7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

List of Figures

List of Tables

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

La scelta delle tecnologie è stata guidata dalla necessità di rispettare i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

2.1 Criteri architetturali e principi di progettazione

Le scelte architetturali del prodotto sono orientate alla separazione delle responsabilita, alla riduzione dell'accoppiamento e alla facilita di evoluzione del sistema. In particolare:

- **Single Responsibility Principle (SRP):** UI, orchestrazione applicativa, accesso ai dati e mapping sono separati in layer distinti.
- **Open/Closed Principle (OCP):** l'estensione di funzionalita avviene prevalentemente tramite nuove implementazioni (es. adapter/gateway/componenti specializzati) senza modificare i contratti esistenti.
- **Liskov Substitution Principle (LSP):** i componenti dipendono da interfacce e token di iniezione, favorendo la sostituibilita delle implementazioni.
- **Interface Segregation Principle (ISP):** le interfacce sono granulari e orientate al caso d'uso, evitando dipendenze inutili.
- **Dependency Inversion Principle (DIP):** i livelli alti dipendono da astrazioni e non da classi concrete, grazie alla Dependency Injection.

Questi principi sono applicati in modo coerente lungo i container *Renderer* e *Main*, con l'obiettivo di garantire testabilita, manutenibilita e robustezza del prodotto.

3 Linguaggio di Programmazione: TypeScript

Scelta	Versione	Motivazione
TypeScript	5.9.2	Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron.

Table 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

3.1 Deployment: Electron

Scelta	Versione	Motivazione
Electron	30.0.0	Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti.

Table 4: Dettagli sulla scelta di Electron come piattaforma di deployment

3.2 Framework Frontend: Angular

Scelta	Versione	Motivazione
Angular	21.2.5	Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice.

Table 6: Dettagli sulla scelta di Angular come framework frontend

3.3 Librerie Aggiuntive

Scelta	Versione	Motivazione
Better SQLite	12.8.0	SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto.
sqlite-vss	0.1.2	Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema.
ONNX Runtime	1.17.3	Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto.
Transformers.js	3.8.1	Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto.

Table 8: Dettagli sulle scelte di librerie aggiuntive

3.4 Tecnologie di analisi statica

Scelta	Versione	Motivazione
SonarQube	2026.2	Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile.
Prettier	2.9.3	Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme.

Table 10: Dettagli sulle scelte di tecnologie di analisi statica

3.5 Tecnologie di analisi dinamica

Scelta	Versione	Motivazione
Vitest	2.3.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock.
Playwright	1.59.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E.

Table 12: Dettagli sulle scelte di tecnologie di analisi dinamica

4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP e il sistema operativo per l'interazione con la stampante e il download dei file.

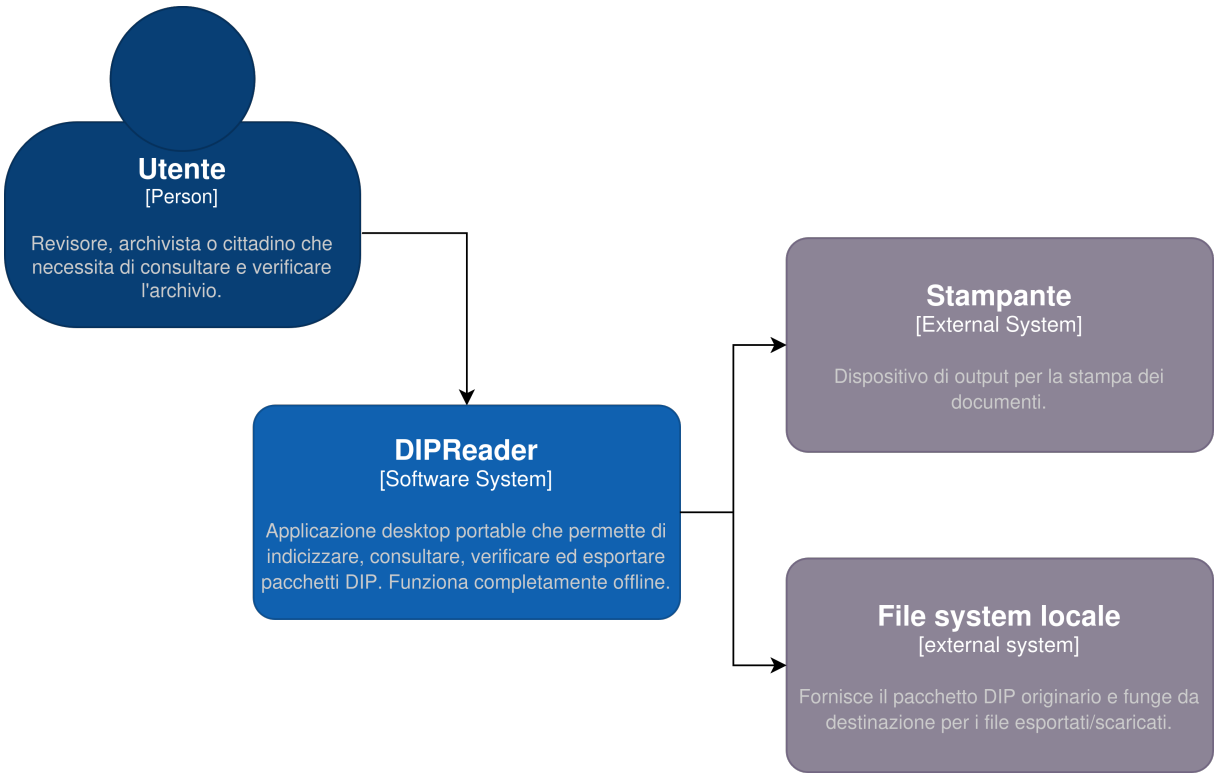

Figure 1: Diagramma C1 - Contesto del sistema *DIPReader*

4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- il sistema operativo per l'interazione con la stampante e il download dei file

5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema cosi progettato consente di avere un backend facilmente estendibile e modificabile, capace di adattarsi alla struttura del DIP. Il frontend risulta disaccoppiato dal backend e organizzato in componenti di presentazione sostituibili, in modo da facilitare evoluzione funzionale e manutenzione.

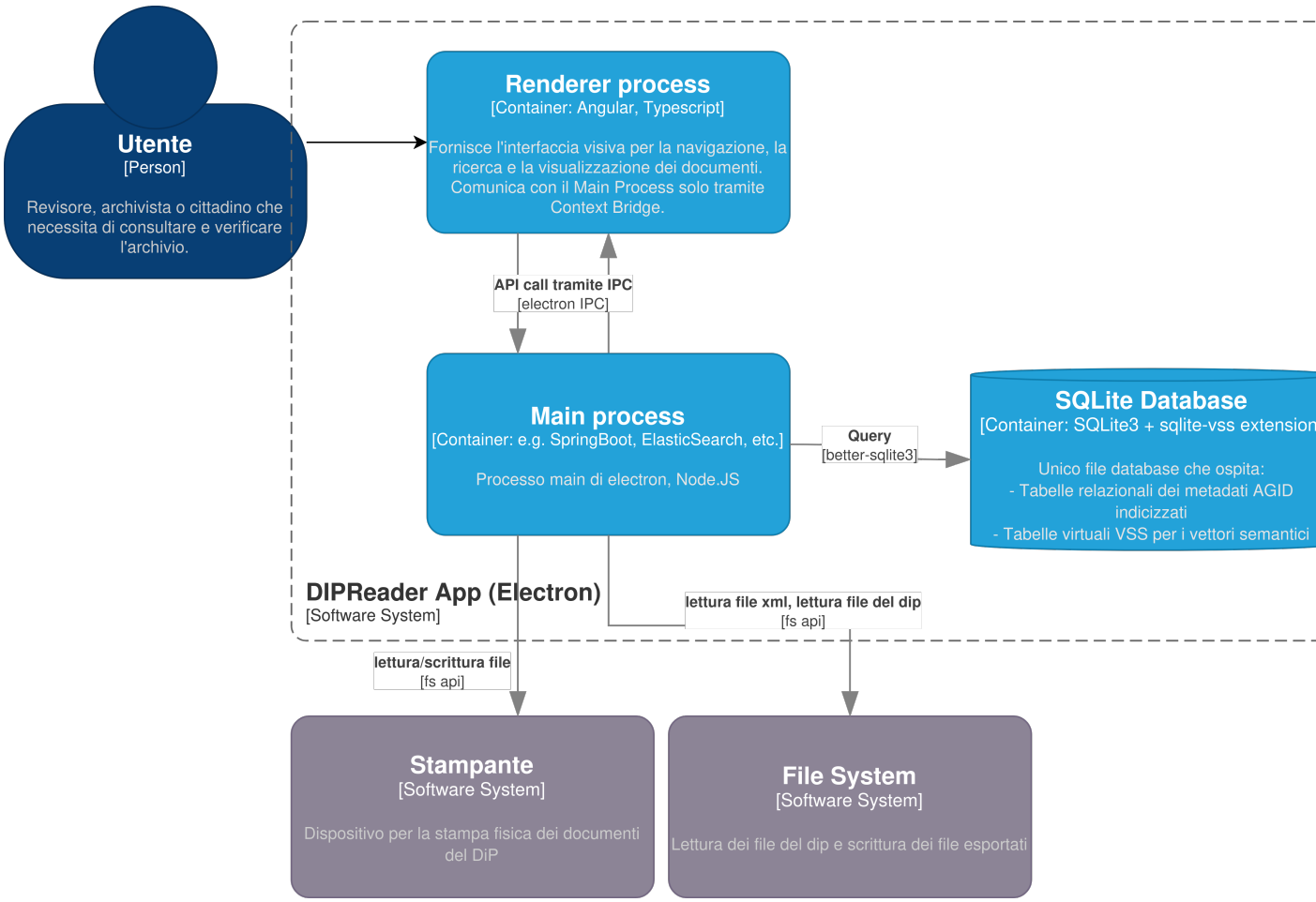

Figure 2: Diagramma C2 - Container del sistema *DIPReader*

5.1 Renderer

5.1.1 Struttura architetturale

Il container *Renderer* è responsabile della presentazione dell'interfaccia utente e dell'interazione con l'utente. Per la gestione dell'interfaccia è stata adottata una struttura a componenti.

Ogni feature espone componenti di presentazione e servizi dedicati al coordinamento della logica applicativa lato UI.
Poiche le interazioni tra i componenti di presentazione sono limitate, ogni componente e mantenuto disaccoppiato dagli altri. I punti di contatto principali sono il routing e i canali IPC, usati per coordinare la navigazione e la comunicazione con il backend.
Per componenti riusabili in piu contesti (ad esempio export e stampa), l'integrazione avviene tramite Dependency Injection, evitando modifiche invasive alla struttura delle feature.
E inoltre applicato il pattern Smart and Dumb Components, che separa la logica di coordinamento dalla presentazione pura.

5.2 Main

5.2.1 Ruolo architetturale

Il container *Main* ospita la logica backend e costituisce il punto di orchestrazione tra IPC, use case applicativi e persistenza su SQLite. Le responsabilita principali sono la registrazione degli handler IPC, la risoluzione delle dipendenze tramite container DI, il coordinamento dei repository/DAO e il bootstrap iniziale dei dati indicizzati.

5.2.2 Stack tecnologico

Componente	**Tecnologia**
Runtime desktop	Electron (main + renderer)
Backend core	TypeScript
Dependency Injection	tsyringe
Database	better-sqlite3
IPC frontend-backend	ipcMain.handle / ipcRenderer.invoke
Architettura	Clean Architecture (layering)

5.2.3 Layer architetturali

1. **IPC Adapter:** boundary verso renderer, gestione canali, mapping DTO.
2. **Use Case:** thin orchestration layer, delega diretta ai repository.
3. **Repository:** astrazione accesso dati, delega ai DAO.
4. **DAO:** query SQL concrete, ricostruzione entity da righe.
5. **Mapper:** trasformazione entity $\leftrightarrow$ DTO.
6. **Entity / Value Objects:** modello di dominio.

5.2.4 Relazione con i principi SOLID

- **SRP:** ogni layer ha responsabilita esplicite e non sovrapposte.
- **OCP:** nuovi use case o adapter possono essere aggiunti estendendo i contratti esistenti.
- **LSP e ISP:** i layer dipendono da interfacce coese e sostituibili.
- **DIP:** il wiring runtime usa token DI e inversione delle dipendenze verso repository/servizi.

5.2.5 Sequenza runtime generale

1. **Renderer invoca IPC:** `window.electronAPI.invoke(channel, payload)`.
2. **Preload autorizza:** verifica prefissi consentiti e inoltra a `ipcRenderer.invoke`.
3. **Main process intercetta:** adapter registra e gestisce l'handler del canale.
4. **Risoluzione DI:** l'adapter risolve il use case richiesto dal container.
5. **Use case delega:** orchestration verso il repository.
6. **Repository delega:** accesso ai dati tramite DAO.
7. **DAO esegue query:** interrogazione SQLite e ricostruzione entity.
8. **Conversione DTO:** mapping entity → DTO al boundary IPC.
9. **Risposta IPC:** il renderer riceve DTO o `null`.

5.2.6 Dependency Injection

Configurazione in `container.ts`:

1. **Provider DB:** `DATABASE_PROVIDER_TOKEN` → `DatabaseProvider`.
2. **Repository tokens:** `*_REPOSITORY_TOKEN` per i vari aggregati.
3. **DAO tokens:** `*_DAO_TOKEN` per le implementazioni concrete.
4. **Use case tokens:** `GET_BY_*` per le operazioni di retrieval.

5.2.7 Bootstrap e indicizzazione

DatabaseBootstrap.ts (avvio):

1. Ricrea database zero.
2. Applica schema.
3. Chiama `IndexDip.execute(dipPath)`.

IndexDip.execute (popolamento DB):

1. Insert: dip.
2. Insert: classi documentali.
3. Insert: processi (con metadati ricorsivi).
4. Insert: documenti (con metadati ricorsivi).
5. Insert: file.

5.2.8 Wiring runtime e registrazioni

In avvio applicazione (`main.ts`) il container runtime esegue:

1. bootstrap database tramite `ApplicationBootstrapAdapter`
2. registrazione token runtime `DIP_PATH_TOKEN`
3. registrazione adapter IPC (es. `BrowsingIpcAdapter`, `CheckIntegrityIpcAdapter`)

Nel bootstrap DB (`DatabaseBootstrap.ts`) viene registrato anche `SQLITE_DB_TOKEN` con la connessione SQLite runtime.

5.2.9 Semantica transazionale condivisa

Le operazioni backend che aggiornano alberi di dati su piu livelli usano `ITransactionManager` con implementazione SQLite (`SqliteTransactionManager`):

1. `BEGIN IMMEDIATE TRANSACTION`
2. esecuzione blocco applicativo
3. `COMMIT` su successo
4. `ROLLBACK` su errore

Questa semantica garantisce consistenza dei dati durante propagazioni gerarchiche di stato.

5.2.10 Sicurezza preload

`preload.ts` consente IPC su prefissi autorizzati:
`ipc:search:`, `browse:`, `file:`, `check-integrity:`, `ipc:indexing:status`.

6 Architettura delle componenti

6.1 [Componente 1]

6.1.1 Descrizione

6.1.2 Interfacce

6.1.3 Dipendenze

6.1.4 Tracciamento dei requisiti

6.2 [Componente 2]

6.2.1 Descrizione

6.2.2 Interfacce

6.2.3 Dipendenze

6.2.4 Tracciamento dei requisiti

6.3 Componenti di Verifica Integrità (Frontend)

6.3.1 Descrizione

La funzionalità di verifica integrità garantisce la correttezza e l'autenticità dei pacchetti DIP, verificando ricorsivamente l'integrità crittografica (generalmente hash SHA-256) dei file contenuti, propagando poi il risultato alla gerarchia documentale.
Per gestire l'albero delle dipendenze documentali sono stati adottati diversi design patterns:

- **Facade Pattern**: tutta la logica dello state management è isolata in *integrity.facade.ts*. In questo modo la facade agisce come single-source-of-truth per tutti i componenti di UI, inoltre coordina le chiamate ai servizi e gestisce lo stato interno *isVerifying* e di invalidazione della cache quando necessario.

- **Angular Signals**: Angular contribuisce con il suo sistema di state management reattivo basato su Signals, che permette di aggiornare automaticamente la UI in risposta ai cambiamenti di stato senza dover programmarne manualmente l'implementazione e limita l'aggiornamento solo ai componenti interessati, migliorando le performance.

- **Gateway Pattern**: come per le altre funzionalità, anche per la verifica integrità è stato adottato il Gateway Pattern per isolare la logica di comunicazione con il backend attraverso *integrity-ipc-gateway.ts*.

- **Smart/Dumb Components**: i componenti di presentazione (dumb) sono responsabili solo della visualizzazione dei dati e dell'emissione di eventi, mentre i componenti smart gestiscono la logica di business e le interazioni con la facade. In questo modo si ottiene una separazione chiara delle responsabilità e una maggiore riusabilità dei componenti di presentazione.

6.3.2 Componente di verifica

Integrity-Dashboard Il componente di presentazione è rappresentato da *integrity-dashboard.component.ts*, che si occupa di visualizzare lo stato della verifica integrità dell'intero DIP e di emettere eventi per avviare la verifica. Attraverso il metodo *loadOverview()* vengono recuperati i dati necessari per visualizzare lo stato del pacchetto, mentre il metodo *startVerification()* avvia la procedura di verifica integrità, delegando alla facade la gestione dello stato e delle chiamate al backend. Mentre la verifica è in corso la proprietà *_isVerifying.set(true)* permette di mostrare un indicatore di caricamento e disabilitare le interazioni con la UI fino al termine del processo. Al termine della verifica, la facade svuota la cache del renderer per poter aggiornare lo stato di verifica dei nodi dell'albero e dei componenti di dettaglio che includono informazioni di integrità.

6.3.3 Componenti di presentazione

Integrity-Summary-Cards Questo *dumb component* include tre card che visualizzano il numero di processi verificati validi, non validi e ancora da verificare.

Integrity-Valid-Panel Questo *dumb component* raccoglie gli elementi verificati, validi e non validi, ponendo la visualizzazione di questi ultimi in cima alla lista. La logica di presentazione degli elementi varia in base al tipo di nodo e allo stato di verifica dei nodi di pari livello, ad esempio se un documento è stato verificato valido la lista lo includerà e rappresenterà come documento, mentre se oltre ad esso sono stati verificati tutti gli altri documenti di pari livello (ovvero dello stesso processo) e sono risultati tutti validi, allora il componente mostrerà un riepilogo del processo invece che dei singoli documenti. In questo modo si ottiene una visualizzazione più compatta e intuitiva dello stato di verifica integrità dell'intero pacchetto. Per i nodi non validi la lista non si limiterà a etichettarli in base al loro livello nell'alberatura documentale, ma mostrerà l'intero percorso del nodo a partire dal livello più alto (cioè la classe documentale) fino al nodo stesso in stile "matrioska", in modo da evidenziare immediatamente la posizione del nodo non valido.

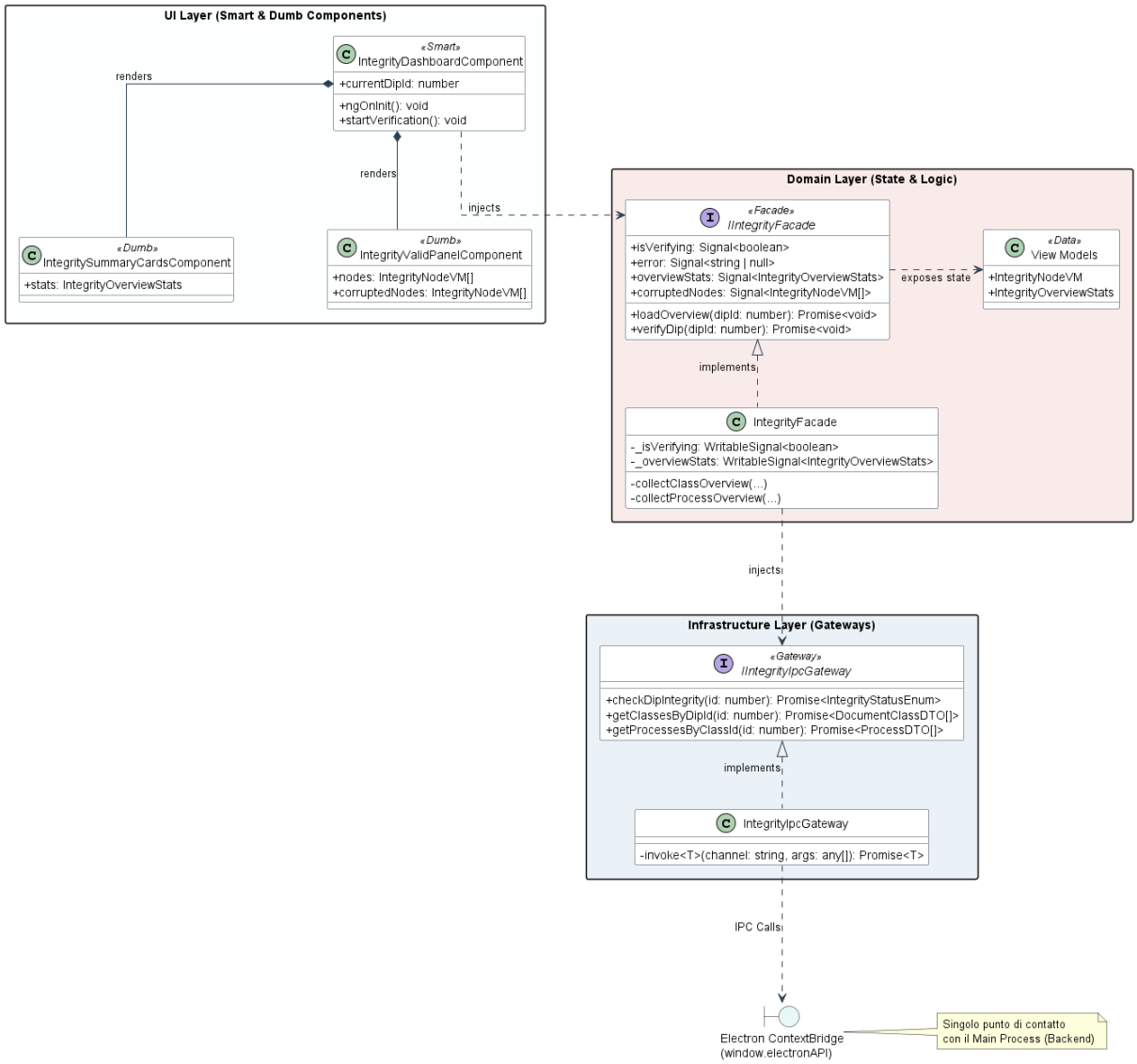

Figure 3: Diagramma delle classi Verifica Integrità (Frontend)

6.4 Componenti di Ricerca (Frontend)

6.4.1 Architettura generale

La sezione di ricerca implementa un'architettura stratificata basata su due pattern fondamentali:

- **Smart & Dumb Components**: separazione netta tra logica (SearchPageComponent) e presentazione (componenti figli).
- **Facade Pattern**: la SearchFacade astrae la complessità dell'orchestrazione dei servizi, fornendo un'interfaccia semplificata al layer di presentazione.

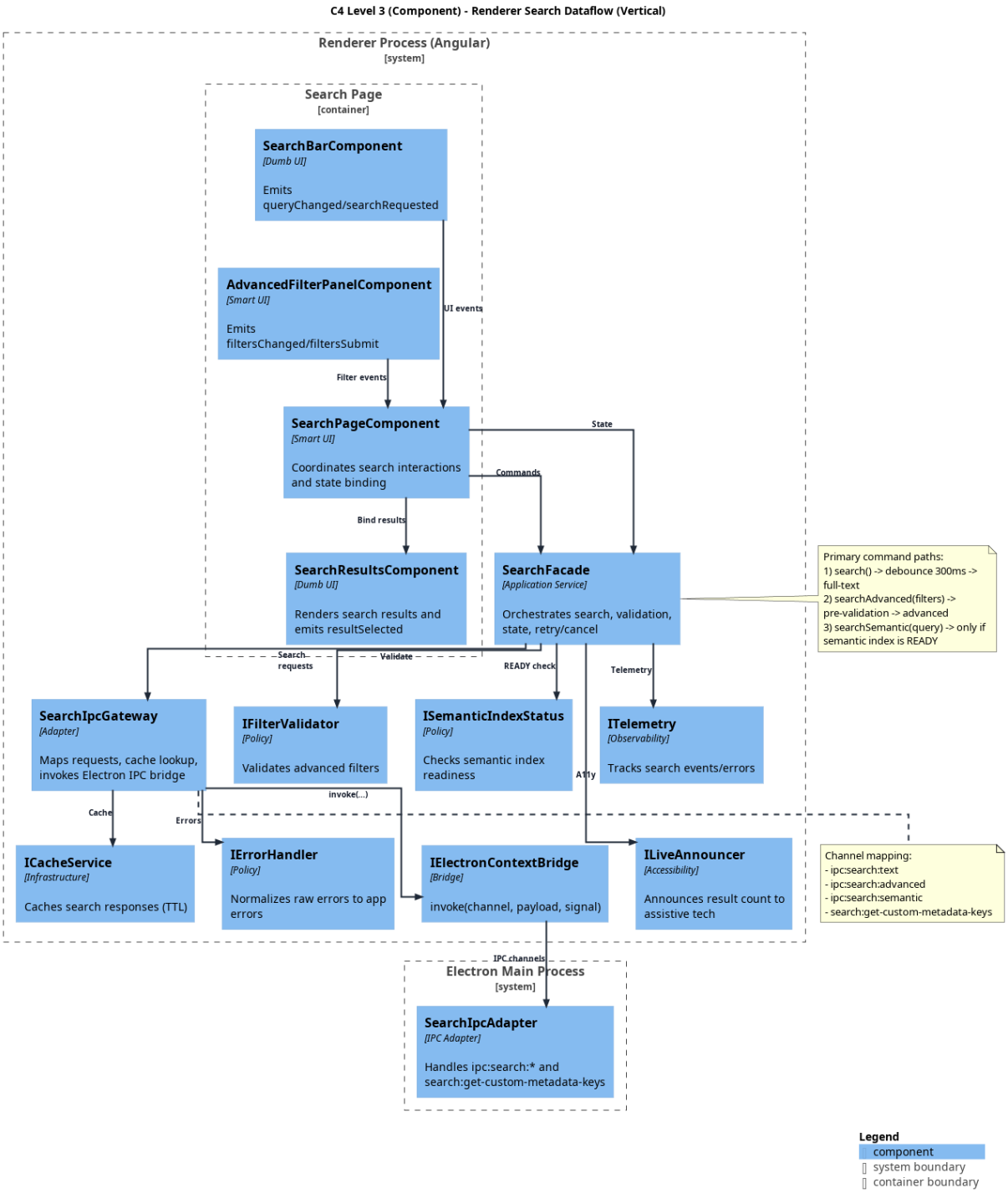

Figure 4: Diagramma C3 - Data flow della funzionalità di ricerca

La SearchPageComponent è il componente radice che:

1. Risolve le dipendenze tramite Angular Dependency Injection (iniettando *interfacce* di servizi, non implementazioni concrete, applicando il principio di sostituzione di Liskov).

2. Delega alla SearchFacade tutta la logica di orchestrazione tra servizi: validazione, gestione dello stato, routing.

3. Espone al template esclusivamente `@Input`/`@Output` properties, mantenendo la componente come contenitore logico puro.

Questo approccio garantisce:

- **Testabilità**: ogni componente e servizio è mockabile in isolamento.
- **Manutenibilità**: i cambiamenti ai servizi non propagano ai componenti di presentazione.
- **Riutilizzabilità**: i componenti dumb possono essere utilizzati in contesti diversi.

6.4.2 Layer di gestione dello stato e validazione

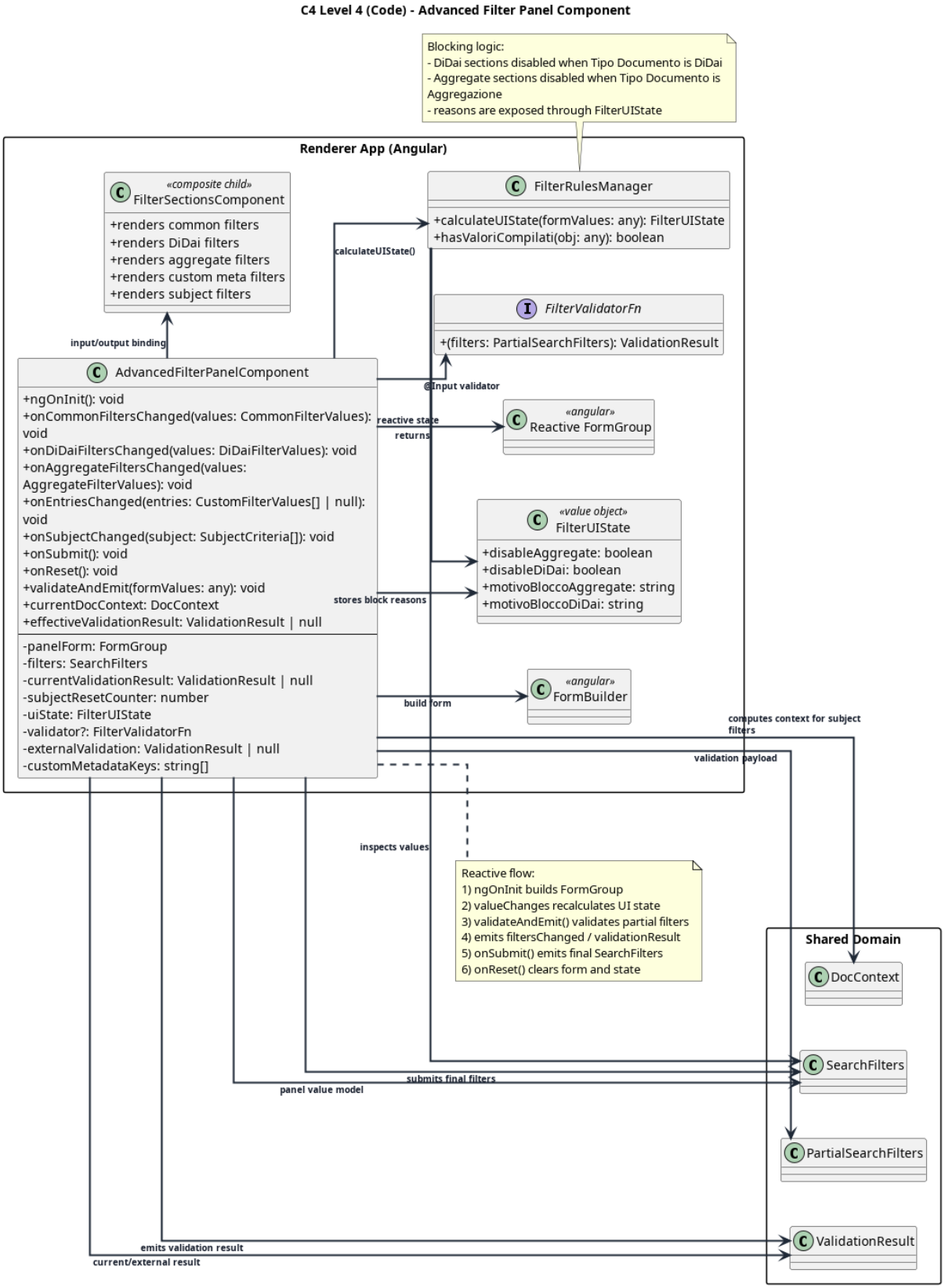

Figure 5: Diagramma C4 - AdvancedFilterPanel e gestione dinamica dei filtri

AdvancedFilterPanelComponent e FilterRulesManager Il pannello filtri avanzati è responsabile della raccolta e gestione dello stato complessivo della ricerca. La sua responsabilità core è:

- Mantenere lo stato mutabile dei filtri durante l'interazione utente.
- Coordinare i sotto-componenti (CommonFiltersComponent, DiDaiFiltersComponent, AggregateFiltersComponent, SubjectsFilterComponent, CustomMetadataComponent).
- Applicare regole di validazione dinamiche tramite il FilterRulesManager.

Il *FilterRulesManager* implementa la logica di validazione contestuale: poiché i campi pertinenti variano in base al tipo di documento selezionato (identificato dal DocContext), questo componente:

- Definisce quali filtri sono visibili, obbligatori o opzionali.
- Applica vincoli di coerenza (ad es., un campo di un tipo specifico non deve essere compilato se quel tipo non e selezionato).
- Puo disabilitare/abilitare dinamicamente i campi senza ricaricare il DOM, garantendo una UX fluida.

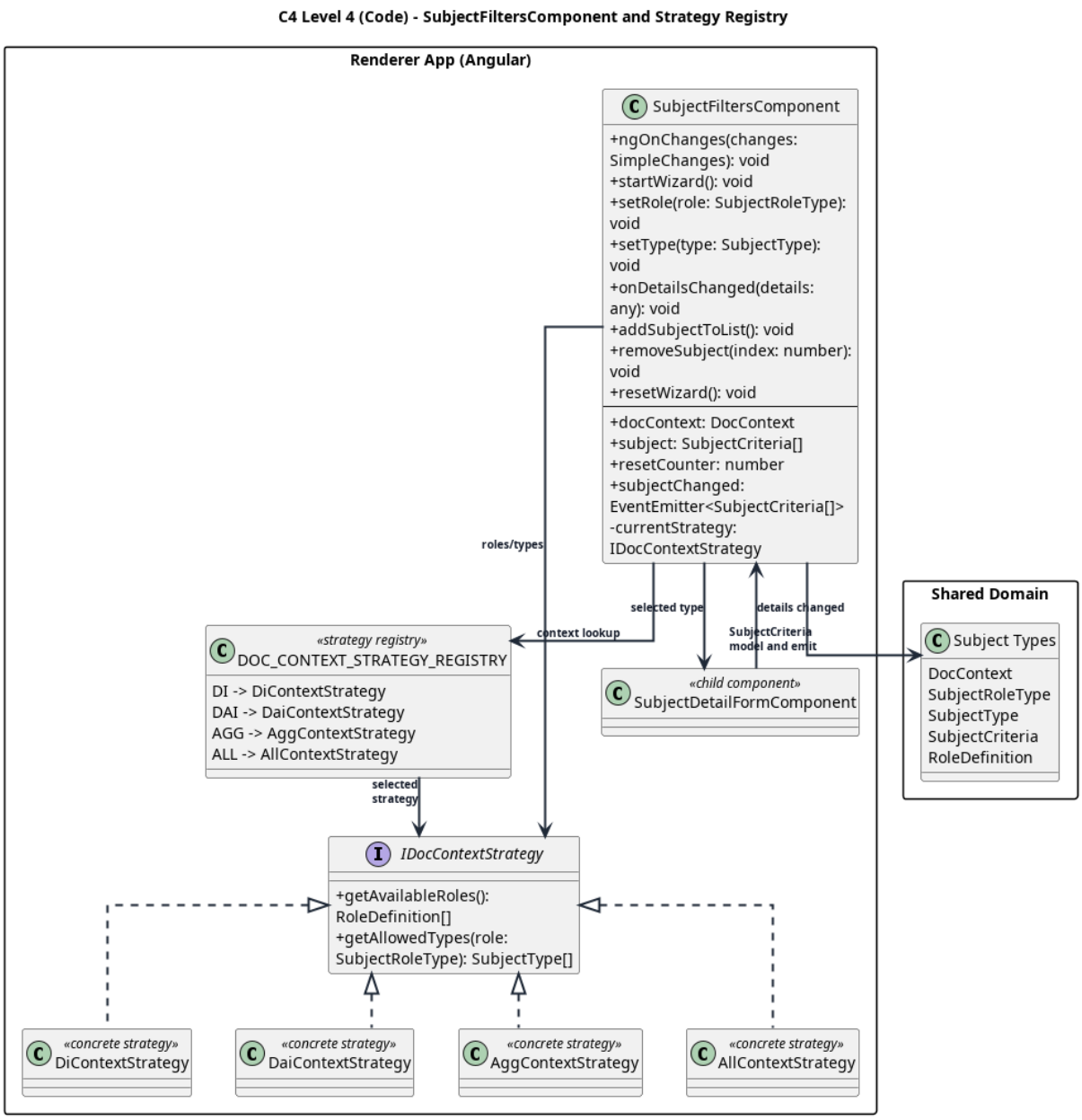

Figure 6: Diagramma C4 - SubjectsFilterComponent e SubjectType Strategy Registry

SubjectsFilterComponent e Strategy Registry Il filtro dei soggetti è strutturato secondo il pattern Strategy per consentire comportamenti specializzati per ogni contesto documentale:

1. **Role Strategy Registry**: in base al DocContext e al tipo di documento, viene selezionata la configurazione di ruoli consentiti. Ad esempio, un documento DiPIndex può avere ruoli (creatore, responsabile) diversi da un'aggregazione (responsabile della procedura, responsabile di fase).

2. **SubjectType Detail Field Registry**: per ogni SubjectType (persona fisica, persona giuridica, ente pubblico), viene mantenuta una registry che associa il tipo ai campi di dettaglio richiesti. Questa consente la generazione *dichiarativa* del form, evitando logiche condizionali sparse nel codice.

3. **Wizard guidato**: il componente orchestra un flusso step-by-step:
    - Selezione del ruolo (reso disponibile dalla Role Strategy).
    - Selezione del SubjectType.
    - Compilazione dinamica dei campi di dettaglio.
    - Raccolta come *SubjectCriteria* e integrazione nello stato globale dei filtri.

In fase di esecuzione della ricerca, i SubjectCriteria vengono trasformati in una struttura gerarchica di query compatibile col back-end, mantenendo una separazione netta tra:

- **Modello di interfaccia** (SubjectCriteria): forma usabile in UI.
- **Modello di ricerca** (QueryModel per back-end): forma serializzabile e ottimizzata per la query.

Ciò consente:

- Facile estensione: nuovi SubjectType o ruoli si aggiungono alle registry senza modificare il componente.
- Manutenzione centralizzata: le regole di validazione e composizione dei filtri risiedono in un'unica registry, non sparse nel codice.
- Coerenza funzionale: un cambiamento nel back-end (nuovo ruolo, nuovo tipo di soggetto) si riflette solo nelle registry, non nella logica del componente.

Componenti di filtro semplici La maggior parte dei filtri sono componenti dumb di presentazione, allineati alle specifiche AGID e alle linee guida Sanmarco Informatica:

CommonFiltersComponent Gestisce i filtri applicabili a **tutti** i tipi di documento:

- Anni di Conservazione (anno inizio, anno fine).
- Chiave descrittiva (campo di testo libero).
- Classificazione con sottocampi (partizione, categoria, codice).

Lo schema è stratificato: la validazione della classificazione (es., "codice" disponibile solo se "partizione" è compilata) è delegata al FilterRulesManager, non al componente.

DiDaiFiltersComponent Gestisce i filtri specifici per Documenti Informatici (DiI) e Documenti Amministrativi Informatici (DAI):

- Tipologia di documento (enum ristretto ai soli tipi DiI/DAI).
- Modalità di formazione (pre-formati, post-formati, cartacei scansionati, ...).
- Nome, versione, ID del documento.
- Dati di formato (tipologia MIME, versione, estensione).

- Verifica e tracciatura: hash algoritmo, hash valore, data tracciatura, modifiche rilevate.

Ciascun campo è memoizzato e reattivo: il componente emette update solo quando il valore effettivamente cambia.

AggregateFiltersComponent Gestisce i filtri specifici per Aggregazioni Documentali (DiP):

- Tipologia aggregazione (fascicolo, procedimento, cartella).
- ID aggregazione.
- Date di apertura e chiusura.
- Dati di procedimento: numero, data inizio, fase corrente.
- Dati di assegnazione: assegnatario, data assegnazione.

CustomMetadataComponent I metadati custom sono coppie chiave-valore personalizzate non previste dallo schema AGID. La loro gestione presenta sfide specifiche:

1. **Discovery dinamico**: le chiavi disponibili devono riflettere l'insieme di chiavi *effettivamente presenti* nei documenti del repository, non uno schema prefissato.
2. **Implementazione**: il componente:
   - Interroga un servizio (GetCustomMetadataKeysUC dal back-end) per recuperare la lista di chiavi popolate.
   - Popola un'autocomplete o una select renderizzata dinamicamente.
   - Consente all'utente di filtrare documenti per chiave e valore.
   - Valida l'input (es., no valori null, lunghezza massima).
3. **Separazione di responsabilità**: il componente UI gestisce *solo* la presentazione; il servizio di ricerca si occupa di trasformare i criteri custom nel modello di query del back-end.

6.4.3 Layer di visualizzazione dei risultati

SearchResultFactory e Dynamic Component Instantiation La visualizzazione dei risultati di ricerca deve adattarsi al tipo di entità ritornata. Poiché processi, classi, aggregazioni e documenti hanno strutture dati e esigenze di presentazione radicalmente diverse, è stato adottato il pattern *Factory* con registry dinamico:

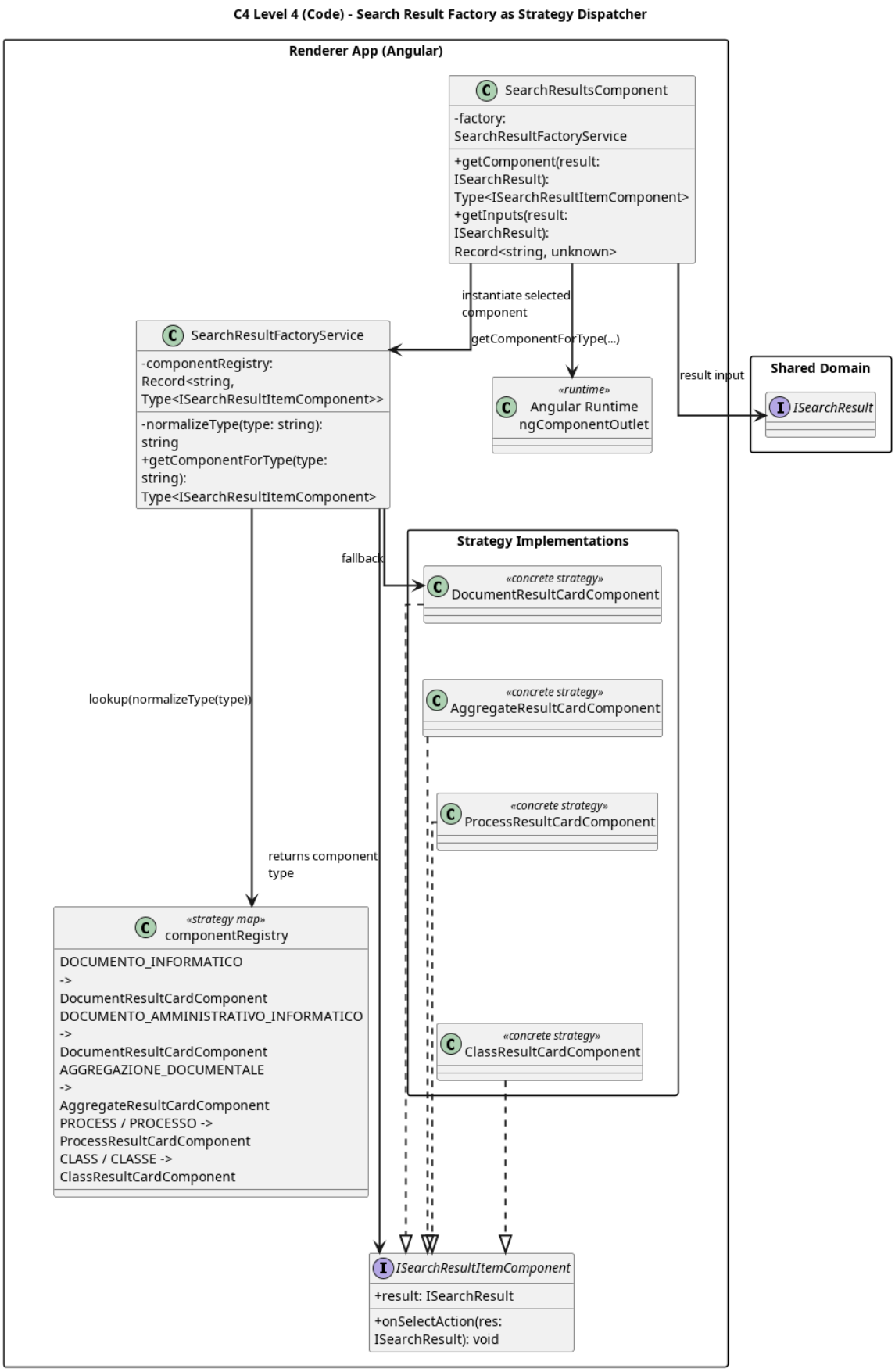

Figure 7: Diagramma C4 - SearchResultFactory e componenti di visualizzazione specializzati

1. **SearchResultFactory**: componente factory che:
   - Interroga il tipo di risultato ricevuto (DocumentResult, ProcessResult, AggregateResult, ClassResult).
   - Consulta una registry configurata *a compile-time* che associa tipo → componente di visualizzazione.
   - Istanzia e proietta il componente corretto nel template tramite *ng-container* e *ngComponentOutlet*.
2. **Componenti di visualizzazione specializzati**:

   ProcessResultComponent Visualizza informazioni essenziali del processo:
   - ID processo (copiabile).
   - Descrizione breve.
   - Numero documenti associati.
   - Link all'apertura del processo.

   No azioni di download/stampa (non applicabili ai processi).

   DocumentResultComponent Visualizza metadati e azioni specifiche per documenti:
   - Anteprima miniatura (se disponibile).
   - Titolo, descrizione, formato.
   - Data creazione, autore (da metadati AGID).
   - Azioni contestuali: apri anteprima, scarica, stampa, mostra metadati.

   AggregateResultComponent Visualizza dati di aggregazione:
   - Tipo aggregazione (fascicolo/procedimento).
   - ID e data apertura.
   - Numero documenti e sotto-aggregazioni.
   - Link all'apertura dell'aggregazione.

   ClassResultComponent Visualizza metadati di classe documentale:
   - Codice e descrizione.
   - Periodo di conservazione.
   - Numero documenti in classe.
3. **Vantaggi dell'approccio**:
   - **Open/Closed**: aggiungere un nuovo tipo di risultato richiede solo di registrare il componente nella registry, senza modificare la factory.
   - **Single Responsibility**: ogni componente di visualizzazione conosce *solo* la struttura del proprio tipo di risultato.
   - **Performance**: solo il componente attivo è renderizzato; gli altri sono completamente distrutti (no lazy loading overhead).

6.4.4 Layer di ricerca globale

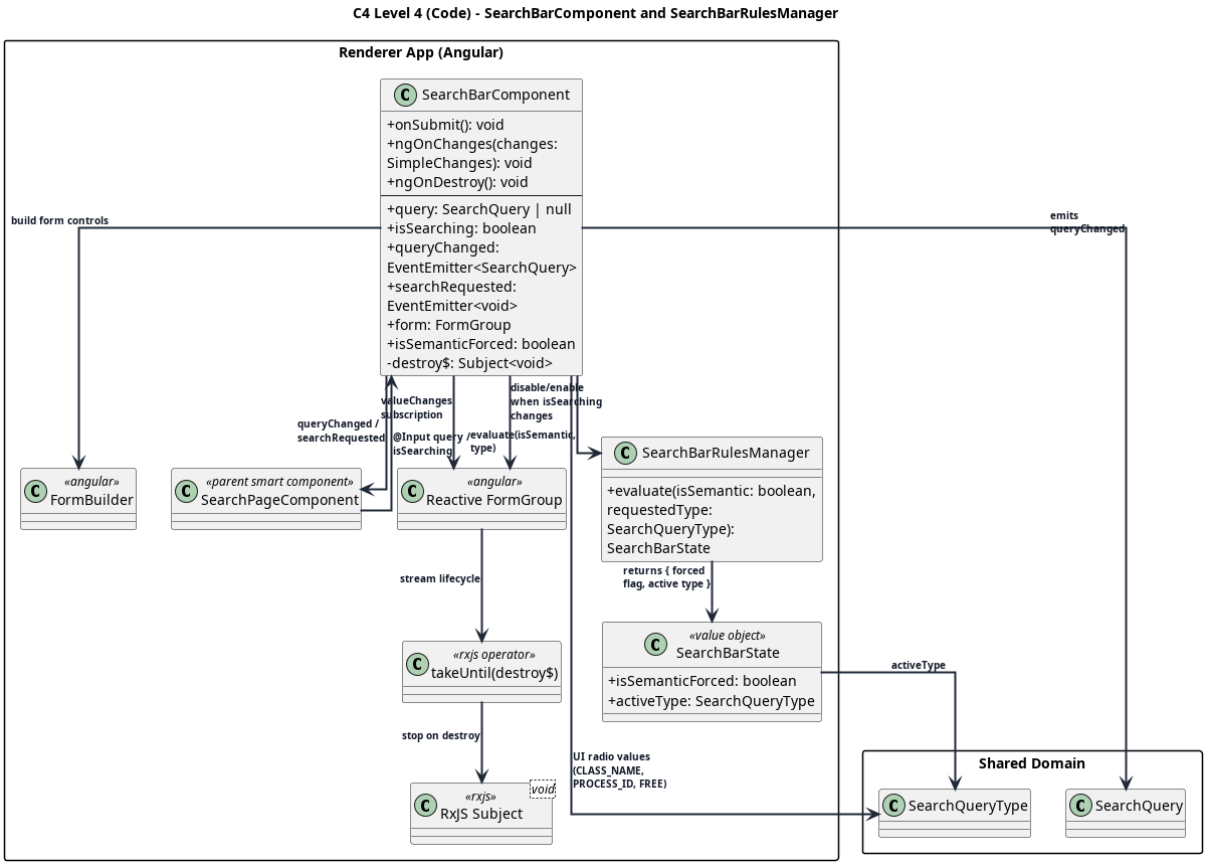

Figure 8: Diagramma C4 - SearchBarComponent e SearchBarRulesManager

SearchBarComponent e SearchBarRulesManager La search bar consente tre modalità di ricerca:

1. **Ricerca per ID**: inserimento dell'ID di processo; ricerca esatta nel back-end.
2. **Ricerca per nome**: inserimento di nome di classe o metadato; ricerca con wildcard.
3. **Ricerca semantica**: ricerca full-text su documenti usando embedding vettoriali (modello Xenova *paraphrase-multilingual-MiniLM-L12-v2*, eseguito nel browser tramite ONNX Runtime).

Il *SearchBarRulesManager* incapsula la logica di:

- Determinazione della modalità di ricerca in base all'input (euristiche: "se input match UUID pattern $\rightarrow$ ricerca ID").
- Abilitazione/disabilitazione della ricerca semantica per tipo di risultato. Attualmente è abilitata *solo* per documenti; estendere a processi richiederebbe configurazione nel servizio, non nel componente.

- Gestione dello stato di loading durante l'embedding vettoriale (che può impiegare millisecondi su CPU).
- Caching dei risultati (opzionale, per evitare query ridondanti).

Il componente di presentazione pura emette eventi:

- onSearch(searchTerm, mode): notifica la ricerca.
- onSemanticToggle(enabled): notifica l'abilitazione/disabilitazione della ricerca semantica.

Questi eventi sono catturati dal SearchPageComponent, che li instrada alla SearchFacade.

6.4.5 Conformità normativa e linee guida

Tutti i componenti di presentazione sono progettati per aderire all'Allegato 5 AGID, in particolare per struttura e denominazione dei metadati, ordine dei campi e obbligatorieta. La validazione della conformità è:

- **Strutturale**: il FilterRulesManager garantisce che solo i campi previsti da Allegato 5 siano disponibili.
- **Dinamica**: il FilterRulesManager può disabilitare campi in base al DocContext senza modificare il template.
- **Testabile**: ogni regola di filtro è una funzione pura, facilmente unit-testabile.

6.4.6 Comunicazione IPC e Gateway Layer

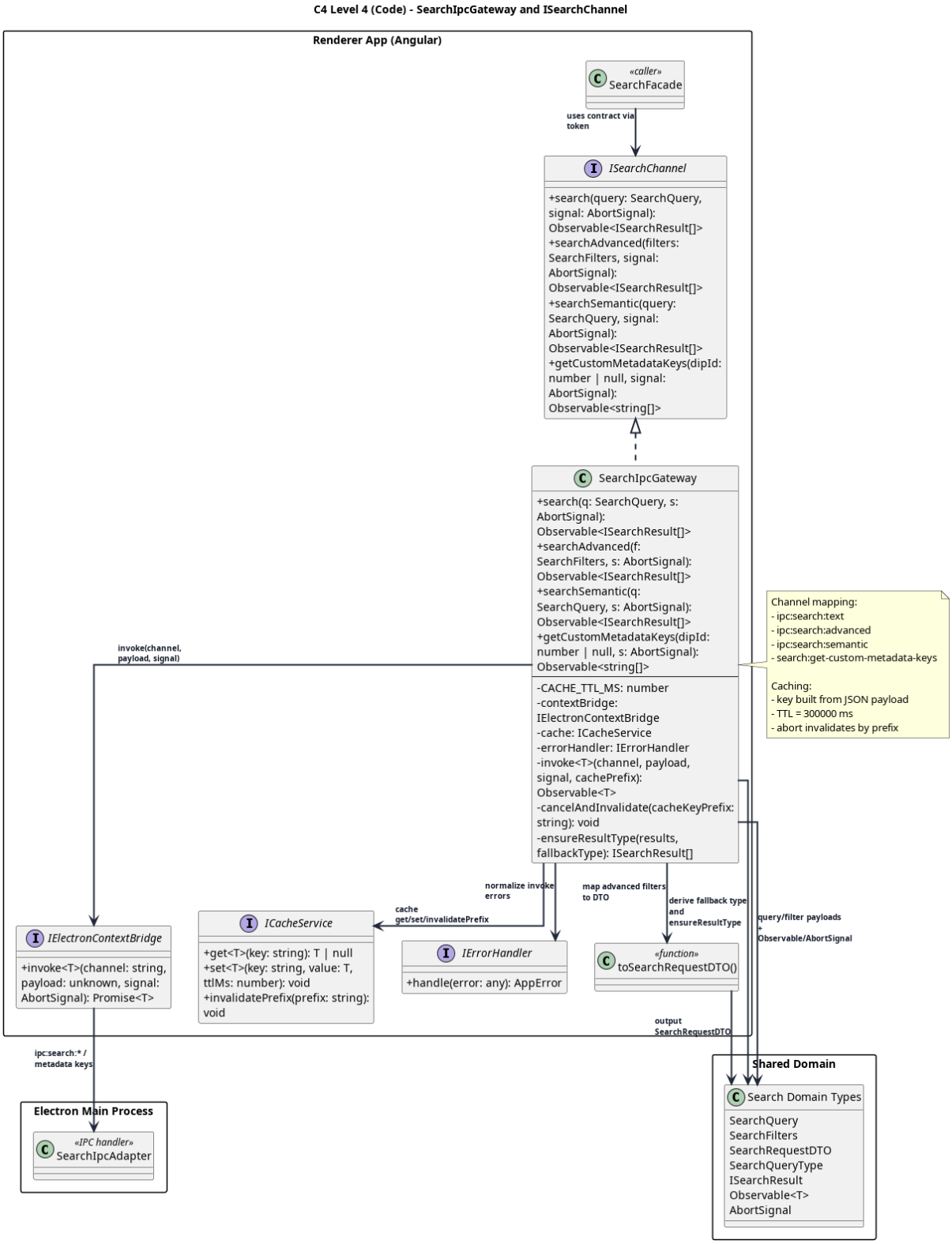

Figure 9: Diagramma C4 - SearchIpcGateway e comunicazione IPC

SearchIpcGateway e ISearchChannel Il SearchIpcGateway implementa ISearchChannel, fungendo da adapter tra il layer UI (SearchFacade) e il back-end tramite Electron IPC.

Responsabilità:

1. **Mapping DTO**: converte SearchQuery/SearchFilters (modello UI) in SearchRequestDTO (modello back-end) tramite toSearchRequestDTO().
2. **Caching**: mantiene cache a TTL 5 minuti per risultati di ricerca identici, riducendo overhead IPC.
3. **Error Handling**: cattura errori IPC e li trasforma in AppError normalizzati per il layer UI.

Interfaccia:

- search(query: SearchQuery, signal: AbortSignal): Observable<ISearchResult[]>
- searchAdvanced(filters: SearchFilters, signal: AbortSignal): Observable<ISearchResult[]>
- searchSemantic(query: SearchQuery, signal: AbortSignal): Observable<ISearchResult[]>
- getCustomMetadataKeys(dipId: number | null, signal: AbortSignal): Observable<string[]>

Flusso IPC

1. SearchFacade richiama gateway.search(query, abortSignal).
2. Gateway:
   (a) Verifica cache con cacheKey = hash(query).
   (b) Se HIT: restituisce cached results.
   (c) Se MISS: invoca contextBridge.invoke(SEARCH_CHANNEL, searchRequestDTO).
   (d) Aspetta risposta dal Main Process.
   (e) Salva in cache se successo.
   (f) Emette risultati via Observable.
3. SearchFacade aggiorna signal<SearchState>.results.
4. SearchResultsComponent visualizza dinamicamente tramite SearchResultFactory.

Integrazione con Core Backend Il backend riceve SearchRequestDTO e lo elabora tramite:

- SearchDocumentsUC: filtra documenti per metadati AGID.
- SearchProcessUC: filtra processi per ID/proprietà.
- SearchSemanticUC: embedding vettoriale (Xenova in browser o backend).
- SearchAggregateUC: filtra aggregazioni per proprietà DiP.

Ognuno restituisce ISearchResult[], che viene serializzato e rimandato al renderer via IPC.

Vantaggi dell'approccio

- **Separation of Concerns**: UI non conosce dettagli back-end, gateway non conosce dettagli UI.
- **Performance**: caching riduce IPC overhead.
- **Testabilità**: gateway è mockabile, backend è testabile in isolamento.
- **Resilienza**: error handling e abort signals gestiscono edge cases.

6.4.7 DTO e Trasformazione Dati

SearchRequestDTO vs SearchQuery La separazione tra SearchQuery (modello UI) e SearchRequestDTO (modello back-end) è fondamentale:

SearchQuery Rappresenta lo stato complessivo della ricerca in UI:

- query text, type, useSemanticSearch
- filters common, diDai, aggregate, subject, customMeta
- Contiene domini tipizzati (es., SubjectCriteria[], FilterField[]).
- Facilmente serializzabile per caching/logging.

SearchRequestDTO Rappresenta la richiesta per il back-end:

- Trasformazione via toSearchRequestDTO(searchQuery, filters).
- Ottimizzata per efficienza back-end: flattened, indici normalizzati.
- Non contiene dati UI-specifici (es., stati di UI, flag di caricamento).
- Utilizzata come chiave di cache nel gateway.

Mappers: SearchRequestMapper e CustomMetadataMapper

- *toSearchRequestDTO()*: converte SearchQuery $\rightarrow$ SearchRequestDTO.
  - Flattens nested filters.
  - Normalizza subject criteria in formato back-end.
  - Aggiunge context headers (es., dipId, userId).
- *DocumentMetadataMapper*: converte raw metadata XML/JSON $\rightarrow$ DocumentDTO.
  - Valida conformità AGID.
  - Estrae metadati standardizzati vs custom.

ISearchResult Union Type Il risultato di ricerca è una discriminated union:

- IDocumentSearchResult type: 'DOCUMENT', id, title, metadata, ...
- IProcessSearchResult type: 'PROCESS', id, docCount, ...
- IAggregateSearchResult type: 'AGGREGATE', id, docCount, ...
- IClassSearchResult type: 'CLASS', code, description, ...

Questa discriminazione permette al SearchResultFactory di istanziare il componente corretto, garantendo type safety.

6.5 Validazione Filtri

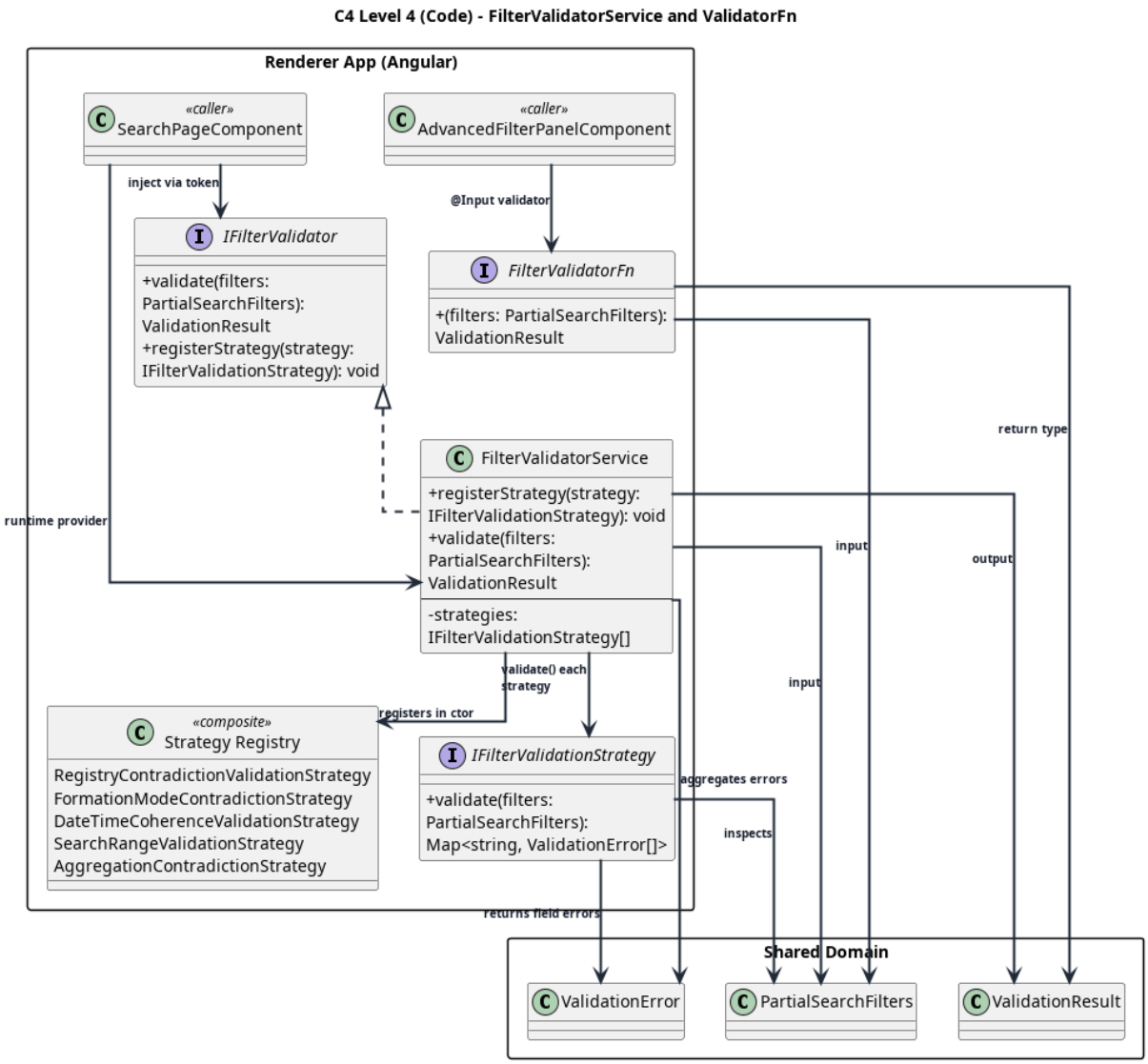

Figure 10: Diagramma C4 - FilterValidatorService e FilterValidatorFn

FilterValidatorService e FilterValidatorFn La validazione non è un aspetto casuale ma un layer dedicato. Il FilterValidatorService applica regole di validazione pure e composabili:

- **Pure Functions**: ogni regola è una funzione pura FilterValidatorFn = (filters) → ValidationResult.
- **Composizione**: regole si combinano (AND/OR) per validazioni complesse.
- **Error Mapping**: ValidationError[] contiene campo, messaggio, severity (ERROR | WARNING).

Esempi di regole:

- Se DiDaiFilters.tipologia è compilata, allora modalitàFormazione è obbligatoria.
- Se SubjectFilter include PERSONA_FISICA, allora codFiscale è obbligatorio.
- Se AggregateFilters.dataApertura è compilata, allora dataChiusura >= dataApertura.
- CommonFilters.annoInizio >= 1900 (vincolo AGID).

Inoltre anche in questo caso l'utilizzo di un pattern Strategy consente di applicare regole specifiche in base ai filtri presi in considerazione. Questo permette di aggiungere nuove regole senza modificare il codice esistente, ma semplicemente aggiungendo nuove funzioni di validazione e registrandole nella registry.

ValidationError e Stato di Errore Lo stato di ricerca mantiene una Map<fieldName, ValidationError> che:

- Permette visualizzazione inline degli errori nei componenti dumb.
- È aggiornata dinamicamente durante la compilazione (non solo al submit).
- È resettata prima di una nuova ricerca.

6.6 Componenti di Dettaglio (Frontend)

6.6.1 Descrizione

All'interno di DIPReader la feature di dettaglio rappresenta il cuore della consultazione. Poiché un DIP aderisce allo standard OAIS, le entità che lo compongono all'interno dell'albero hanno una struttura ben definita non sempre trasversale a tutte le entità: processi, documenti informatici, documenti amministrativi informatici, aggregazioni documentali e le classi documentali che fungono da contenitori logici. La feature di dettaglio è quindi composta da più componenti, ognuna dedicata alla visualizzazione di una specifica entità, che condividono però un'interfaccia comune per poter essere utilizzata in modo intercambiabile all'interno dell'albero.

Invece di duplicare le pagine di routing per ogni entità quindi è stata sviluppata la feature *Item-Detail* per sfruttare una singola rotta generica */detail/:itemType/:itemId* che, a seconda del tipo di entità e dell'identificativo, renderizza il componente specifico per quella entità. Questo approccio implementa un design pattern di tipo strategy con polimorfismo statico, in cui la logica di selezione dell'entità da visualizzare è centralizzata e ben definita grazie alla tipizzazione statica di TypeScript, la scelta è stata dettata dalla considerazione che la struttura dei dati è ben definita e non si prevede l'aggiunta di nuovi tipi di entità in futuro, la possibilità di estensione riguarda invece i metadati delle entità.

6.6.2 Componente principale

Il componente principale della feature è *ItemDetailPageComponent*, che agisce da orchestratore e si occupa di:

- multiplexing dei Token delle facade delle singole entità e indirizzamento alla facade corretta, con un componente di fallback per coprire i casi di entità vuote come le classi documentali;
- adattamento del layout grafico a seconda del tipo di entità, per i documenti viene utilizzato un layout comprensivo di un'anteprima del file, mentre per aggregazioni e processi viene mostrato un indice dei documenti che li compongono;

6.6.3 Componente dei metadati

Il componente *MetadataPanelComponent* decide quali componenti di metadati visualizzare in base al tipo di entità selezionata. Esso intercetta i dati passati dall'interfaccia dell'entità da visualizzare e in base ad essa decide quali componenti mostrare. Ad esempio, per i documenti sono presenti i componenti relativi alle informazioni del processo di conservazione, degli allegati, del formato del file, dei metadati custom, ecc., mentre per le aggregazioni e i processi vengono mostrati oltre ai metadati anche gli indici dei documenti che li compongono.

6.6.4 Componente di azioni sugli elementi

Il componente *DocumentActionsComponent* crea la barra orizzontale di azioni che è possibile eseguire su un'entità, per tutte offre la verifica dell'integrità, per i documenti è anche possibile scaricarli o stamparli tramite il componente della feature di Export.

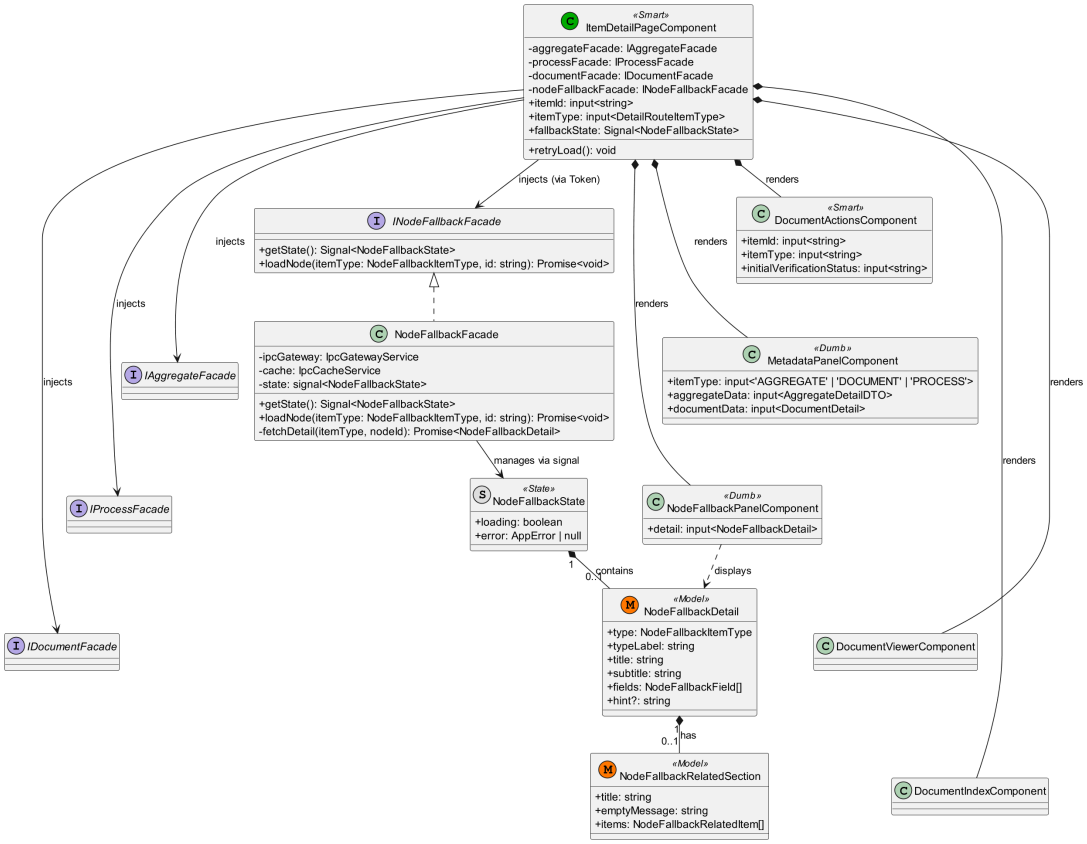

Figure 11: Diagramma delle classi tecnico della feature di dettaglio - struttura generale

6.6.5 Componenti specifici per entità - DOCUMENT

È il modulo più denso di informazioni, ospita una notevole quantità di componenti *dumb* per le sezioni specifiche dei metadati OAIS e i componenti per la visualizzazione dell'anteprima.

DocumentViewerComponent Il componente *DocumentViewerComponent* è responsabile della visualizzazione dell'anteprima del documento, supporta i formati PDF, XML, JPEG, PNG e testo semplice. Se il formato del documento non è supportato, interviene la logica di gestione degli errori unificata che restituisce un *AppError*.

Schede metadati Ogni sottoinsieme dei metadati nel formato XML viene visualizzato in un componente dedicato:

- *document-metadata*: identificativo, livello di riservatezza, hash del documento;
- *aip-info*: fornisce le proprietà di storicizzazione a lungo termine e riporta le informazioni di conservazione a norma;
- *attachments*: mostra l'elenco degli allegati associati al documento, con la relativa impronta crittografica;
- *change-tracking*: proprietà di versionamento;

- *classification-info*: restituisce l'indice e il piano di classificazione secondo i registri della PA;
- *format-info*: mostra il formato MIME del documento;
- *registration-data*: mostra i dati di registrazione del documento, sia protocollato che non protocollato;
- *subject-list*: mostra i soggetti coinvolti nella produzione del documento, con i relativi ruoli;
- *verification-info*: mostra i risultati dell'ultima verifica di integrità eseguita sul documento;
- *custom-metadata*: mostra i metadati custom definiti dall'ente, con un formato standard chiave-valore.

Interfacce/Dominio/Servizi/Mappers Come per tutte le feature, sono presenti le interfacce per i servizi, i servizi stessi, i mappers per adattare i dati restituiti dal backend al formato utilizzato dai componenti e le interfacce per i DTO estrate dagli schema XSD forniti e dalla documentazione AGID sui metadati.
Rimangono rispettati quindi i principi di separazione dei compiti, single responsibility e inversion of control.

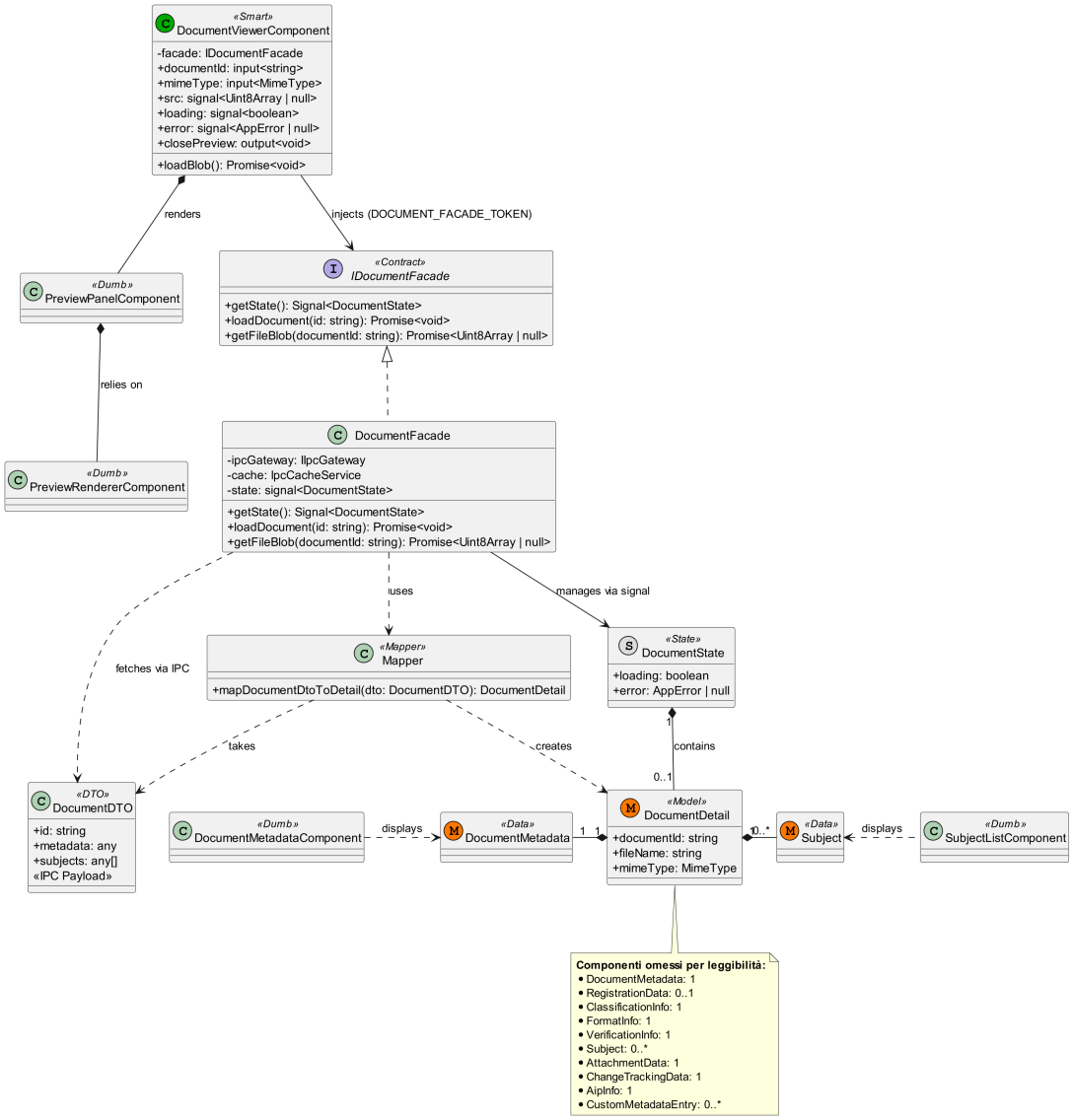

Figure 12: Diagramma delle classi tecnico della feature di dettaglio - entità Document

6.6.6 Componenti specifici per entità - AGGREGATE

AggregateFacade Come per i documenti, è presente una facade dedicata per le aggregazioni che si occupa di orchestrare i servizi necessari a recuperare i dati e i metadati dell'entità (con collegamento ai canali IPC). Inoltre implementa un sistema di *Fetch Retry Synchronization* per gestire situazioni in cui i dati non sono ancora disponibili al momento della richiesta.

Componenti *dumb* Per le aggregazioni sono presenti componenti dedicati alla visualizzazione dei metadati specifici:

- *admin-procedure*: indica il procedimento a cui il fascicolo appartiene, nonché lo stato di avanzamento e le relative fasi;
- *aggregate-metadata*: mostra i metadati specifici dell'aggregazione, come Tipo,

Progressivo, Classificazione, ecc.;

- *aggregate-subjects*: mostra i soggetti coinvolti nella produzione dell'aggregazione, con i relativi ruoli, differiscono leggermente dai soggetti dei documenti;
- *document-index*: mostra l'indice dei documenti che compongono l'aggregazione, con collegamento alla relativa pagina di dettaglio.

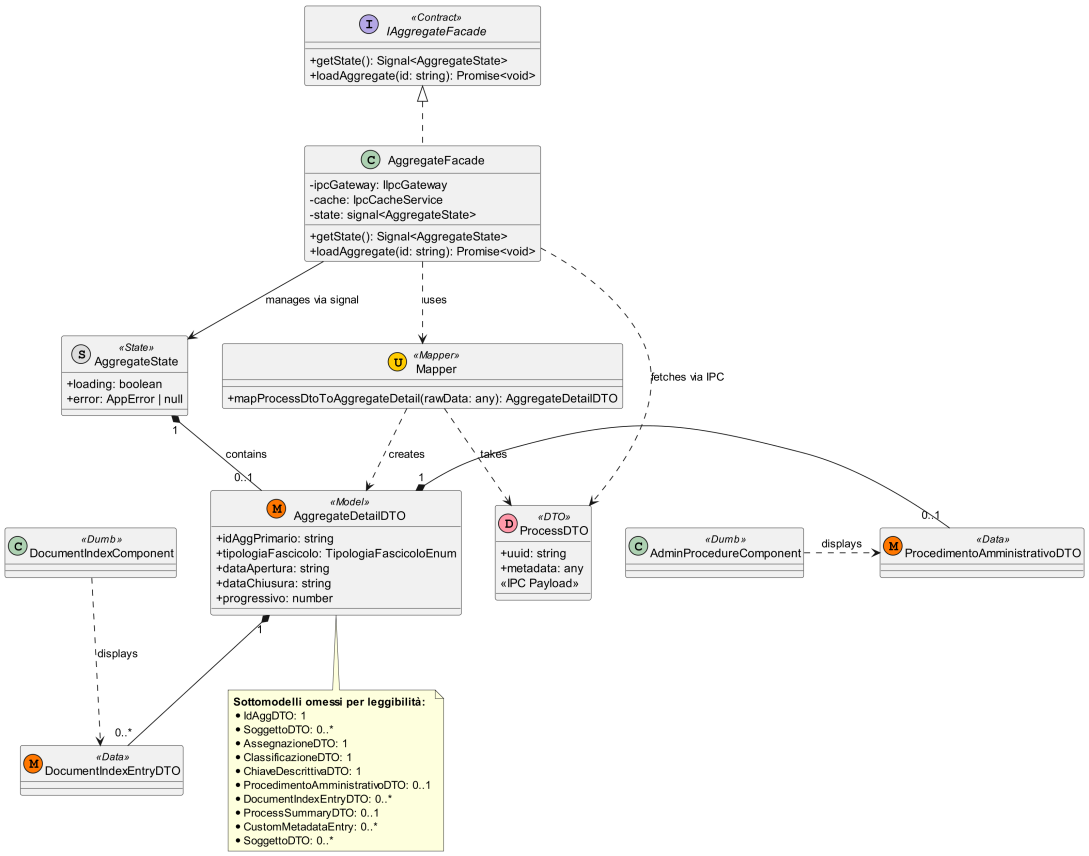

Figure 13: Diagramma delle classi tecnico della feature di dettaglio - entità Aggregate

6.6.7 Componenti specifici per entità - PROCESS

ProcessFacade Come per le aggregazioni, è presente una facade dedicata per i processi che si occupa di orchestrare i servizi necessari a recuperare i dati e i metadati dell'entità (con collegamento ai canali IPC). Anche in questo caso implementa un sistema di *Fetch Retry Synchronization*.

Componente di presentazione *Process-Metadata-Component* in questo caso è l'unico componente specifico per i processi, mostra i metadati specifici dell'entità con un formato *card-based*, lo stato di verifica e informazioni sulla classe documentale di appartenenza.

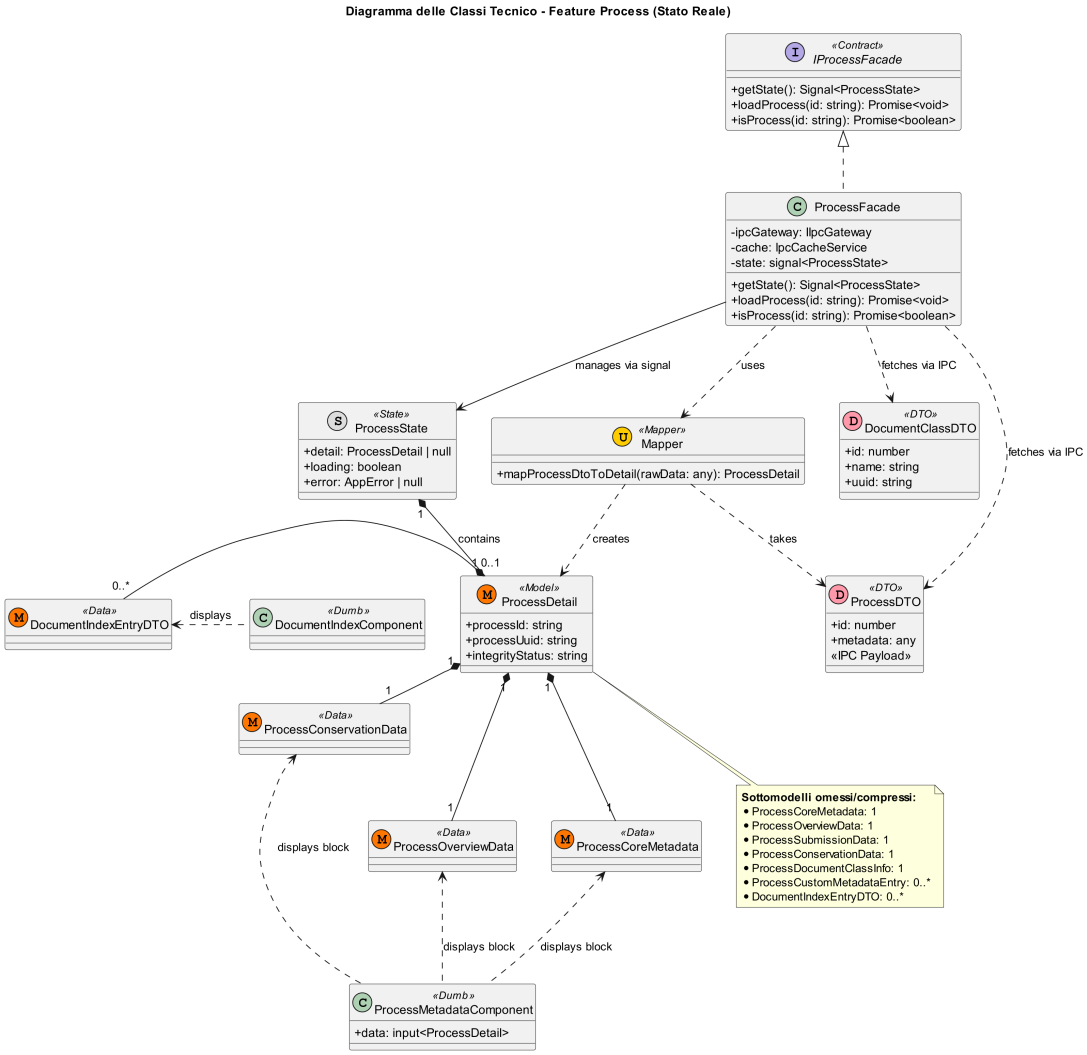

Figure 14: Diagramma delle classi tecnico della feature di dettaglio - entità Process

7 Backend - Browsing

7.1 Panoramica

Navigazione gerarchica

Il modulo Browsing implementa la navigazione gerarchica dei dati indicizzati in SQLite: $DIP_G \rightarrow$ Classi documentali $\rightarrow$ Processi $\rightarrow$ Documenti $\rightarrow$ File. Supporta inoltre getter alternativi per ID e stato di integrità.

Le caratteristiche backend trasversali al container Main (stack tecnologico, layering, sequenza runtime generale, Dependency Injection, bootstrap e sicurezza preload) sono descritte in **??**.

7.1.1 Organizzazione file backend

Layer	File
Entry point IPC	`main.ts`, `preload.ts`, `ipc-channels.ts`,`BrowsingIpcAdapter.ts`
Use case	`dip/*`, `classe-documentale/*`,`process/*`, `document/*`, `file/*`
Repository	`I*Repository.ts`, `*Repository.ts`
DAO	`I*DAO.ts`, `*DAO.ts`, `mappers/*Mapper.ts`
Entity / DTO	`entity/*`, `dto/*`, `value-objects/*`
Database	`schema.sql`, `DatabaseBootstrap.ts`, `IndexDip.ts`

7.1.2 Flusso end-to-end

Flusso gerarchico di navigazione.

BROWSE_GET_DIP_BY_ID
↓
BROWSE_GET_DOCUMENT_CLASS_BY_DIP_ID
↓
BROWSE_GET_PROCESS_BY_DOCUMENT_CLASS
↓
BROWSE_GET_DOCUMENTS_BY_PROCESS
↓
BROWSE_GET_FILE_BY_DOCUMENT

Contratto IPC definito in `ipc-channels.ts` e registrato in `BrowsingIpcAdapter.ts`:

Canale	Payload	Return	Use case
BROWSE_GET_DOCUMENT_BY_ID	id: number	DocumentDTO | null	IGetDocumentByIdUC
BROWSE_GET_DOCUMENTS_BY_PROCESS	processId: number	DocumentDTO[]	IGetDocumentByProcessUC
BROWSE_GET_DOCUMENTS_BY_STATUS	status: IntegrityStatusEnum	DocumentDTO[]	IGetDocumentByStatusUC
BROWSE_GET_FILE_BY_ID	id: number	FileDTO | null	IGetFileByIdUC
BROWSE_GET_FILE_BUFFER_BY_ID	id: number	Buffer | null	IGetFileByIdUC + lettura fs
BROWSE_GET_FILE_BY_DOCUMENT	documentId: number	FileDTO[]	IGetFileByDocumentUC
BROWSE_GET_FILE_BY_STATUS	status: IntegrityStatusEnum	FileDTO[]	IGetFileByStatusUC
BROWSE_GET_PROCESS_BY_ID	id: number	ProcessDTO | null	IGetProcessByIdUC
BROWSE_GET_PROCESS_BY_STATUS	status: IntegrityStatusEnum	ProcessDTO[]	IGetProcessByStatusUC
BROWSE_GET_PROCESS_BY_DOCUMENT_CLASS	documentClassId: number	ProcessDTO[]	IGetProcessByDocumentClassUC
BROWSE_GET_DOCUMENT_CLASS_BY_DIP_ID	dipId: number	DocumentClassDTO[]	IGetDocumentClassByDipIdUC
BROWSE_GET_DOCUMENT_CLASS_BY_STATUS	status: IntegrityStatusEnum	DocumentClassDTO[]	IGetDocumentClassByStatusUC
BROWSE_GET_DOCUMENT_CLASS_BY_ID	id: number	DocumentClassDTO | null	IGetDocumentClassByIdUC
BROWSE_GET_DIP_BY_ID	id: number	DipDTO | null	IGetDipByIdUC
BROWSE_GET_DIP_BY_STATUS	status: IntegrityStatusEnum	DipDTO[]	IGetDipByStatusUC

Table 15: Contratto IPC del modulo Browsing

Nota: Il canale `BROWSE_GET_DIP_BY_DOCUMENT_CLASS` è definito in `ipc-channels.ts` ma non registrato in `BrowsingIpcAdapter`.

7.1.3 Diagrammi architetturali (C3/C4)

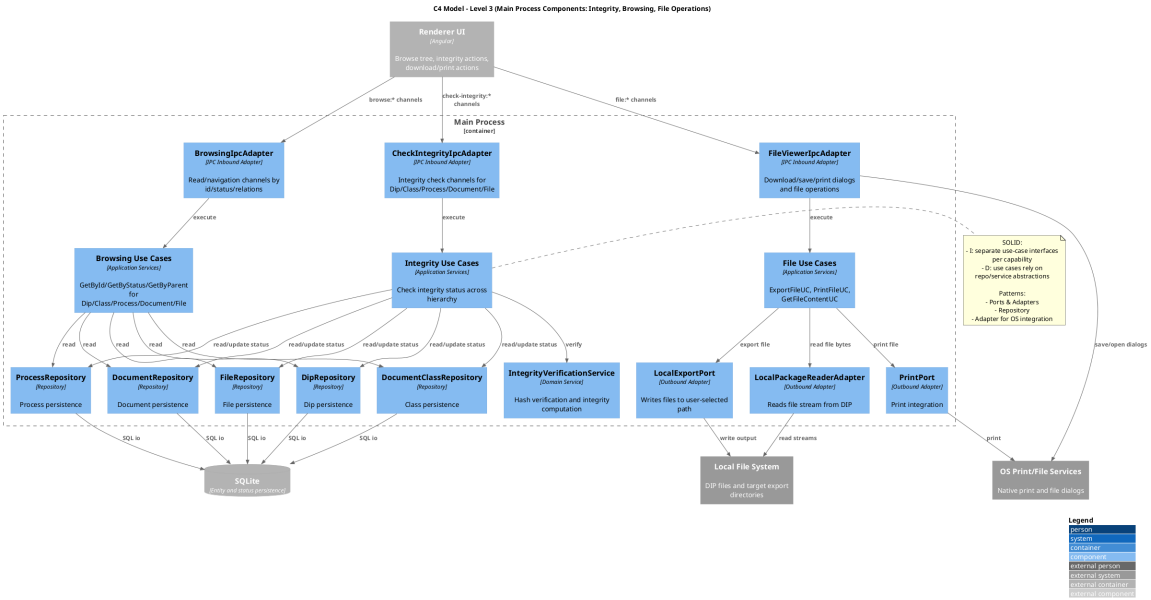

Figure 15: C3 del backend: orchestrazione condivisa tra Browsing e Verifica Integrita

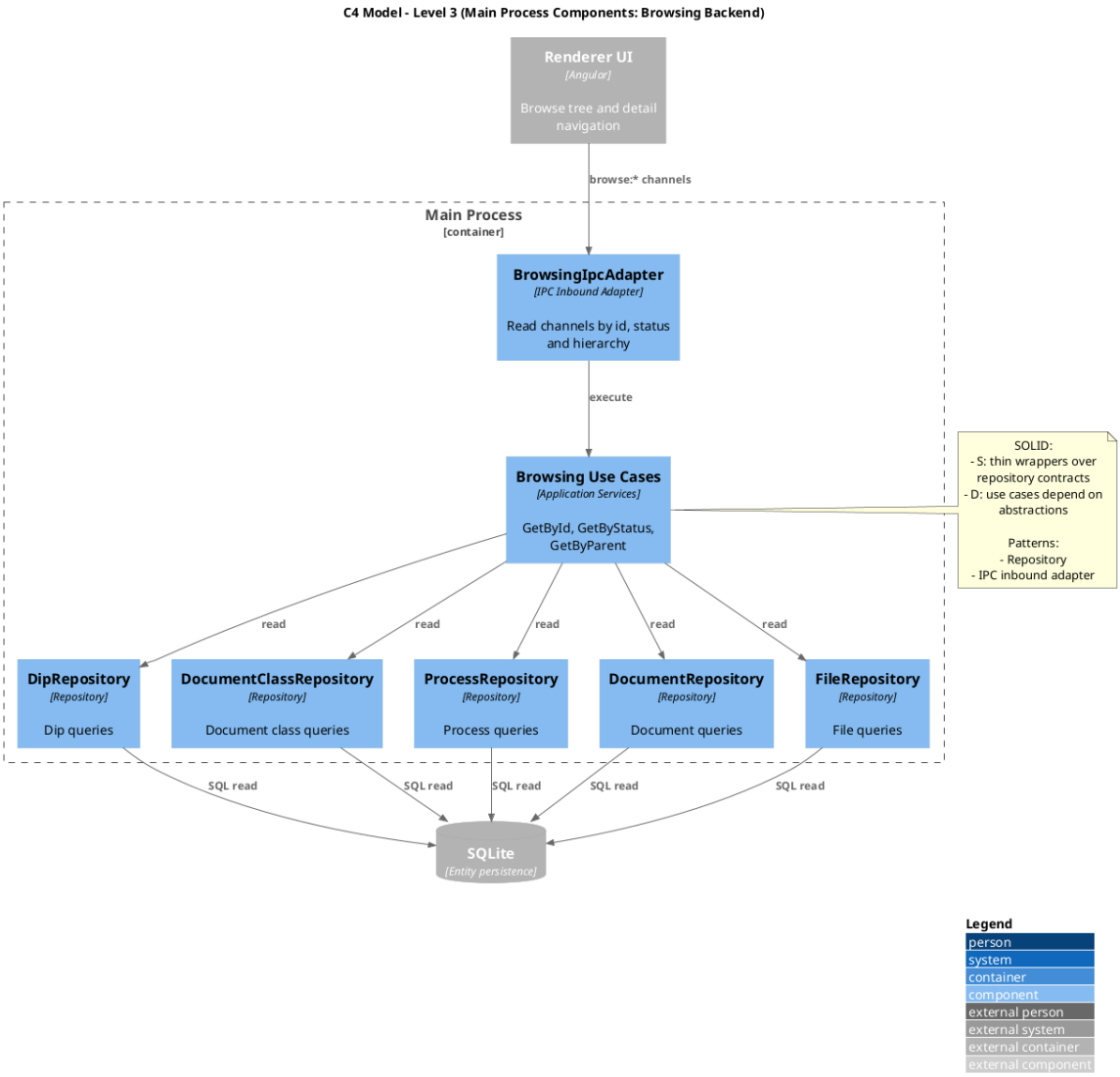

Figure 16: C3 del backend: focus sul flusso Browsing (BrowsingIpcAdapter)

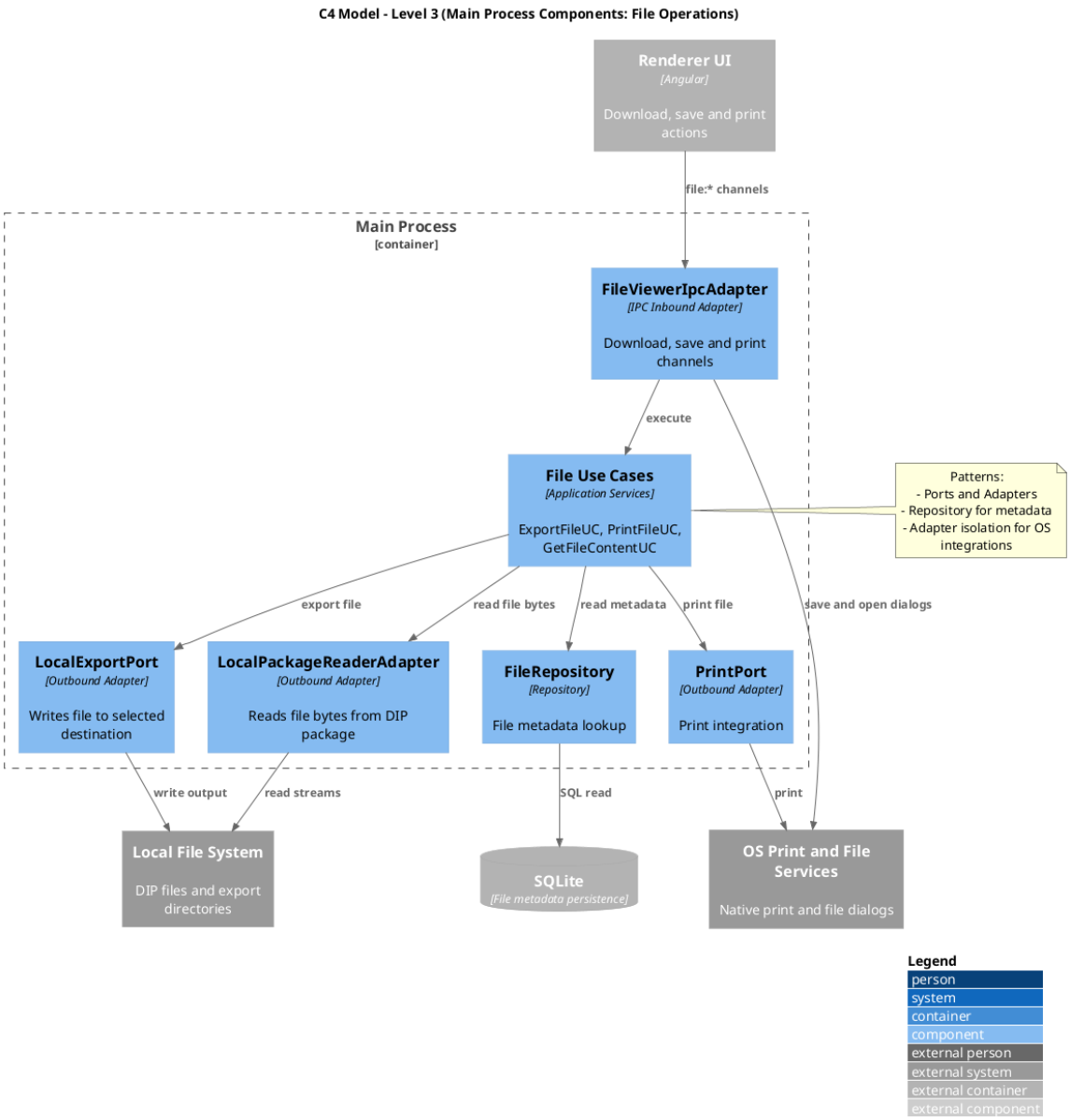

Figure 17: C3 del backend: focus sulle operazioni file (FileViewerIpcAdapter)

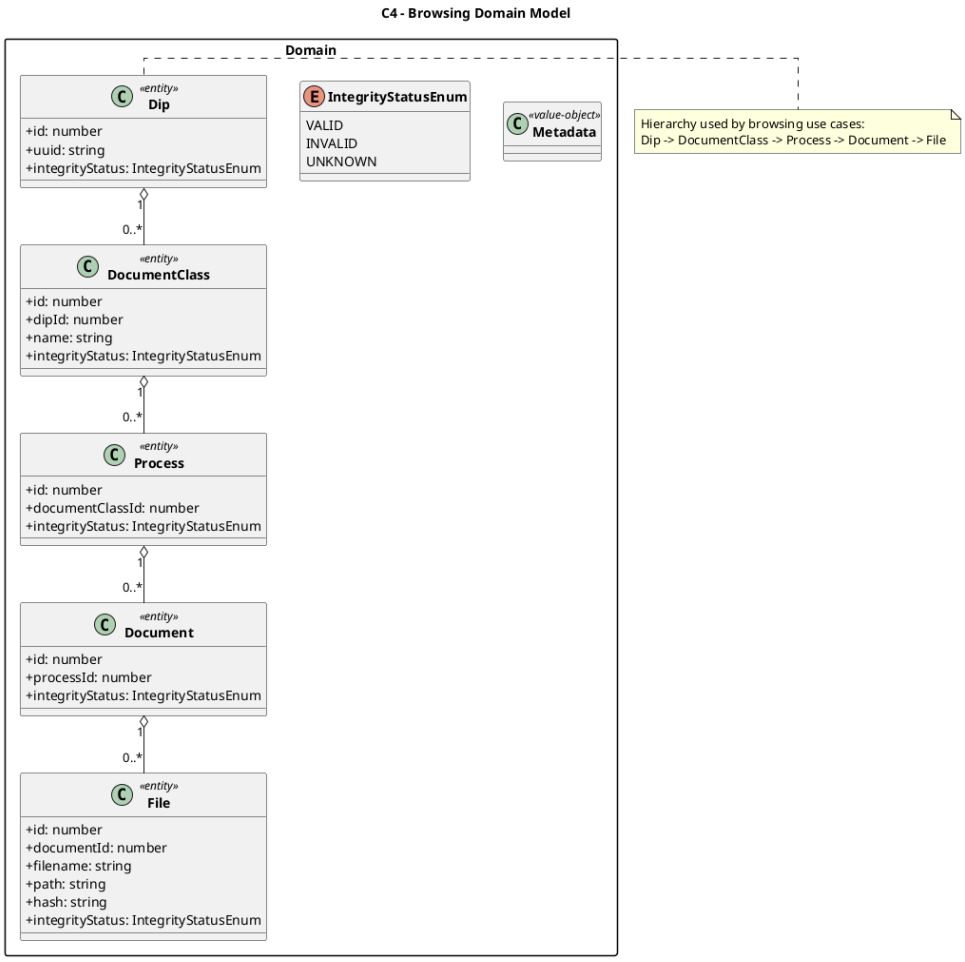

Figure 18: C4 del modulo Browsing - modello di dominio

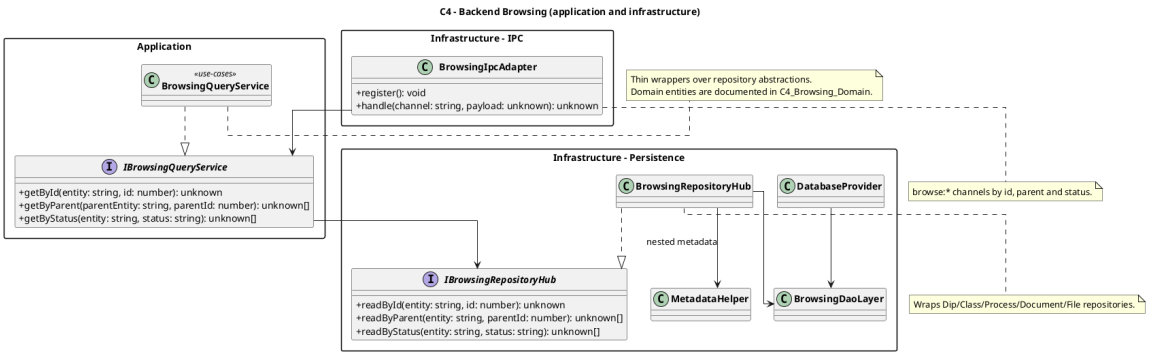

Figure 19: C4 del modulo Browsing - architettura applicativa e infrastrutturale

7.1.4 Use Case: firme getter

Caratteristica della layer

Gli use case sono thin wrapper senza logica aggiuntiva; delegano direttamente al repository corrispondente.

Use Case	Firma
IGetDipByIdUC	execute(id: number): Dip | null
IGetDipByStatusUC	execute(status: IntegrityStatusEnum): Dip[]
IGetDocumentClassByDipIdUC	execute(dipId: number): DocumentClass[]
IGetDocumentClassByIdUC	execute(id: number): DocumentClass | null
IGetDocumentClassByStatusUC	execute(status: IntegrityStatusEnum): DocumentClass[]
IGetProcessByDocumentClassUC	execute(documentClassId: number): Process[]
IGetProcessByIdUC	execute(id: number): Process | null
IGetProcessByStatusUC	execute(status: IntegrityStatusEnum): Process[]
IGetDocumentByIdUC	execute(id: number): Document | null
IGetDocumentByProcessUC	execute(processId: number): Document[]
IGetDocumentByStatusUC	execute(status: IntegrityStatusEnum): Document[]
IGetFileByIdUC	execute(id: number): File | null
IGetFileByDocumentUC	execute(documentId: number): File[]
IGetFileByStatusUC	execute(status: IntegrityStatusEnum): File[]

Interfacce per aggregato.

7.1.5 Layer Repository

Ogni repository delega al DAO corrispondente il retrieval dei dati.

Repository	Implementazione	Metodi getter
IDipRepository	DipRepository.ts	ById, ByUuid, ByStatus
IDocumentClassRepository	DocumentClassRepository.ts	ById, ByDipId, ByStatus
IProcessRepository	ProcessRepository.ts	ById, ByDocumentClassId, ByStatus
IDocumentRepository	DocumentRepository.ts	ById, ByProcessId, ByStatus
IFileRepository	FileRepository.ts	ById, ByDocumentId, ByStatus

Inoltre: `save`, `updateIntegrityStatus` per persistenza; `searchDocument*`, `getIndexedDocumentsCoun` per DocumentRepository.

7.1.6 Layer DAO

Le interfacce `I*DAO` espongono metodi paralleli ai repository per il retrieval e query concrete eseguite in SQLite.

DAO	File	Query / Operazioni
DipDAO	DipDAO.ts	By id/uuid/status; UPSERT uuid
DocumentClassDAO	DocumentClassDAO.ts	By id/dipId/status; search LIKE; UPSERT
ProcessDAO	ProcessDAO.ts	By id/documentClassId/status; metadati albero
DocumentDAO	DocumentDAO.ts	By id/processId/status; search avanzate; ricerca vettoriale
FileDAO	FileDAO.ts	By id/documentId/status; sort is_main DESC
MetadataHelper	MetadataHelper.ts	Load/save ricorsivo; ricostruzione albero

7.1.7 Entity e Value Objects

Modello di dominio. Ogni entity espone getter e `setIntegrityStatus`:

Entity	Attributi principali
Dip	id,uuid,integrityStatus
DocumentClass	id,dipId,dipUuid,uuid,name,timestamp,integrityStatus
Process	id,documentClassId,documentClassUuid,uuid,metadata,integrityStatus
Document	id,uuid,metadata,processId,processUuid,integrityStatus
File	id,uuid,filename,path,hash,isMain,documentId,documentUuid, integrityStatus

7.1.8 Data Transfer Objects

Contratto dati IPC (restituiti dall'adapter al renderer):

DTO	Attributi
DipDTO	id,uuid,integrityStatus
DocumentClassDTO	id,dipId,uuid,name,timestamp,integrityStatus
ProcessDTO	id,documentClassId,uuid,integrityStatus,metadata: MetadataDTO
DocumentDTO	id,processId,uuid,integrityStatus,metadata: MetadataDTO
FileDTO	id,documentId,filename,path,hash,integrityStatus,isMain
MetadataDTO	name,value (scalare|MetadataDTO[]),type: MetadataType

Table 21: DTO schema (metadati caricati ricorsivamente)

Nota: Il converter genera eccezioni se `id` o FK sono `null` durante la conversione.

7.1.9 Mapping e conversioni

Trasformazioni tra layer di persistenza, entity e DTO:

Conversione	File	Funzioni
Entity → DTO	`EntityToDtoConverter.ts`	`*ToDto` per dip, docClass, process, document, file, metadata
SQL → Entity	`*Mapper.ts`	`*Mapper` per dip, docClass, process, document, file, metadata

`MetadataMapper.fromFlatRows` ricostruisce l'albero da righe flat con controllo cicli/root.

7.1.10 Comportamenti runtime

Semantica getter

By-ID: restituisce `null` se entity non trovata
List: restituisce array, anche vuoto
DTO: conversione applicata nel boundary IPC prima della restituzione

Lettura file buffer (`BROWSE_GET_FILE_BUFFER_BY_ID`).

1. Recupera file entity tramite `IGetFileByIdUC`
2. Risolve percorso assoluto: `path.resolve(dipPath, file.getPath())`
3. Legge contenuto con `fs.readFileSync`
4. In caso di errore I/O: log e ritorno `null`

8 Backend - Verifica Integrita

8.1 Panoramica

Verifica gerarchica dello stato

Il modulo Verifica Integrita esegue controlli su 5 livelli gerarchici: File → Documento → Processo → Classe documentale → DIP. Il controllo puo essere avviato da qualsiasi livello e propaga lo stato nel sottoalbero interessato.

8.1.1 Perimetro funzionale

La verifica opera sui seguenti livelli:

1. **File**
2. **Documento**
3. **Processo**
4. **Classe documentale**
5. **DIP**

Ogni invocazione calcola lo stato del nodo root e dei suoi discendenti, con persistenza progressiva in SQLite tramite `updateIntegrityStatus(...)`.

8.1.2 Organizzazione file backend

Layer	File
Entry point IPC	`main.ts`, `preload.ts`, `ipc-channels.ts`,`CheckIntegrityIpcAdapter.ts`
Use case	`*/ICheck*IntegrityStatusUC.ts`, `*/impl/Check*IntegrityStatusUC.ts`
Service	`IntegrityVerificationService.ts`, `HashingService.ts`
Transaction manager	`SqliteTransactionManager.ts`
Repository	`I*Repository.ts`, `*Repository.ts`
DAO	`I*DAO.ts`, `*DAO.ts`
Entity / DTO	`entity/*`, `dto/*`
Database	`schema.sql` (colonna `integrity_status` su 5 tabelle)

Table 24: Organizzazione modulo Verifica Integrita

8.1.3 Flusso end-to-end

Sequenza di verifica.

1. Il renderer invoca il canale `check-integrity:*`
2. `CheckIntegrityIpcAdapter` intercetta l'handler e risolve il use case
3. Il use case delega a `IntegrityVerificationService`
4. Il service esegue `runInTransaction(...)`
5. Il service verifica hash file e aggrega stati sui parent
6. Il service persiste gli stati sui livelli coinvolti
7. L'handler IPC ritorna `IntegrityStatusEnum`

Flusso gerarchico di propagazione.

CHECK_DIP_INTEGRITY_STATUS
↓
CHECK_DOCUMENT_CLASS_INTEGRITY_STATUS
↓
CHECK_PROCESS_INTEGRITY_STATUS
↓
CHECK_DOCUMENT_INTEGRITY_STATUS
↓
CHECK_FILE_INTEGRITY_STATUS

Contratto IPC definito in `ipc-channels.ts` e registrato in `CheckIntegrityIpcAdapter.ts`:

Canale	Payload	Return	Use case
CHECK_FILE_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckFileIntegrityStatusUC
CHECK_DOCUMENT_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckDocumentIntegrityStatusUC
CHECK_PROCESS_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckProcessIntegrityStatusUC
CHECK_DOCUMENT_CLASS_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckDocumentClassIntegrityStatusUC
CHECK_DIP_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckDipIntegrityStatusUC

Table 25: Contratto IPC del modulo Verifica Integrita

Nota: Il canale `INTEGRITY_VERIFY` e definito in `ipc-channels.ts` ma non risulta registrato in `CheckIntegrityIpcAdapter`.

8.1.4 Diagrammi architetturali (C3/C4)

Il flusso backend condiviso con il modulo Browsing e rappresentato in Figura **??**. La vista focalizzata sul percorso di verifica integrita e mostrata in Figura **??**.

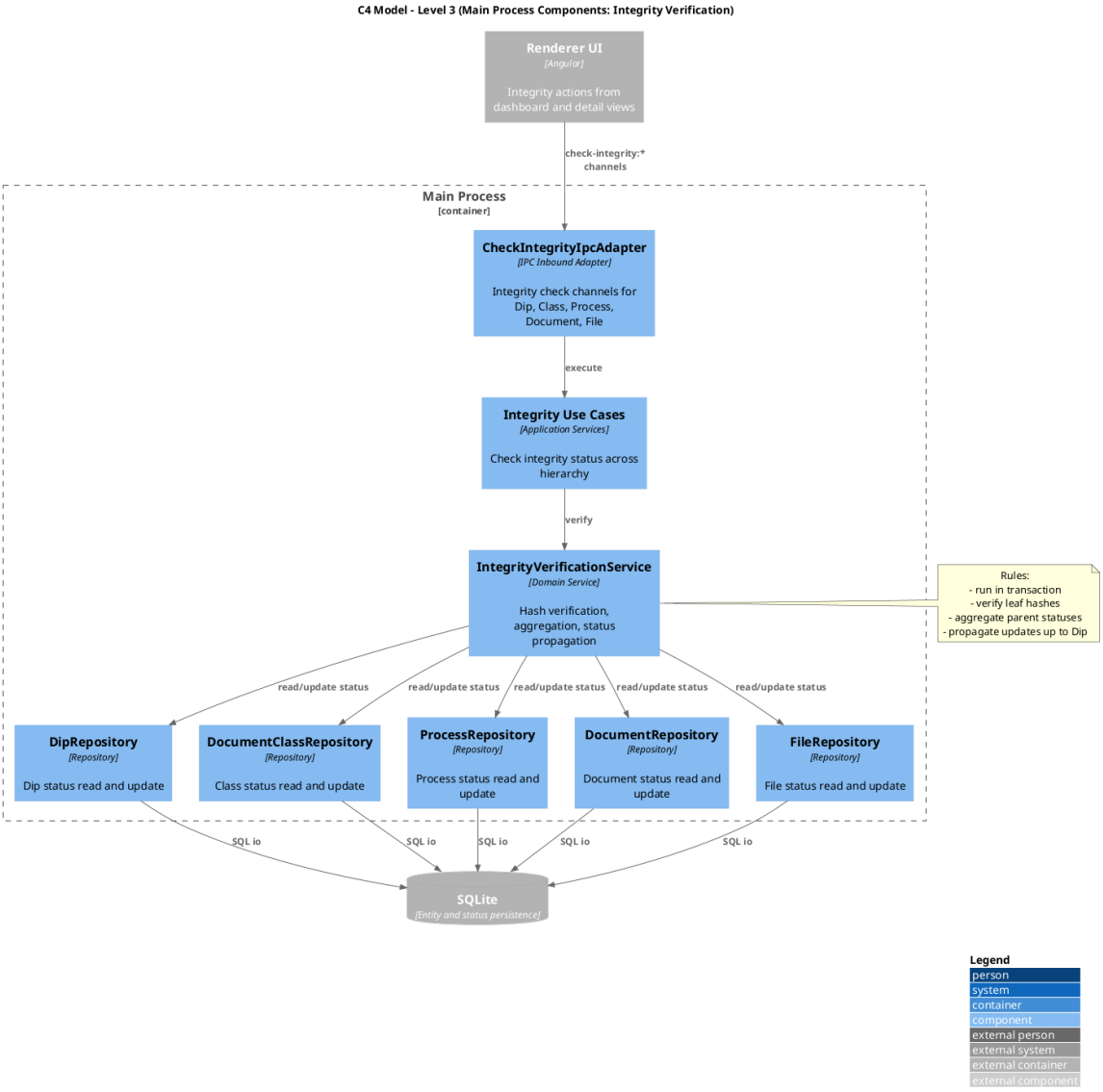

Figure 20: C3 del backend: focus sulla verifica integrita (CheckIntegrityIpcAdapter)

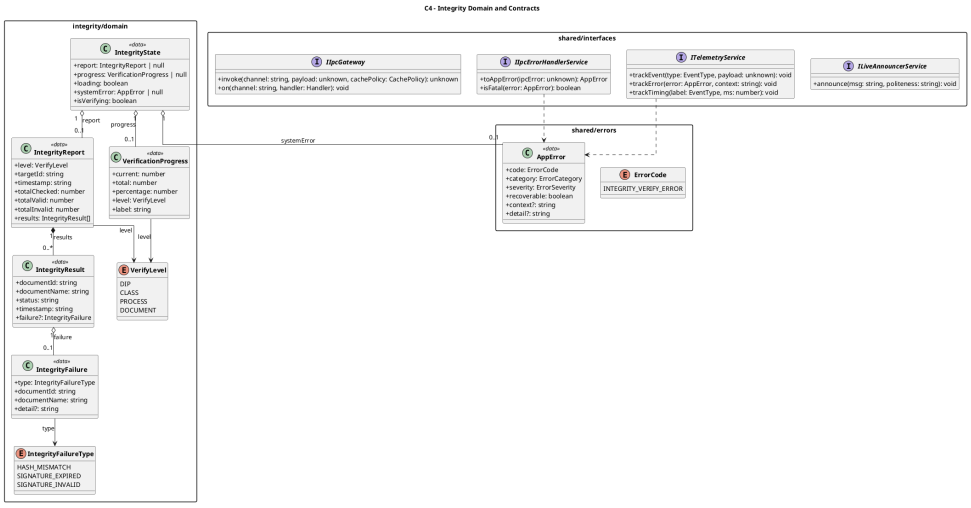

Figure 21: C4 del modulo Verifica Integrita - dominio e contratti

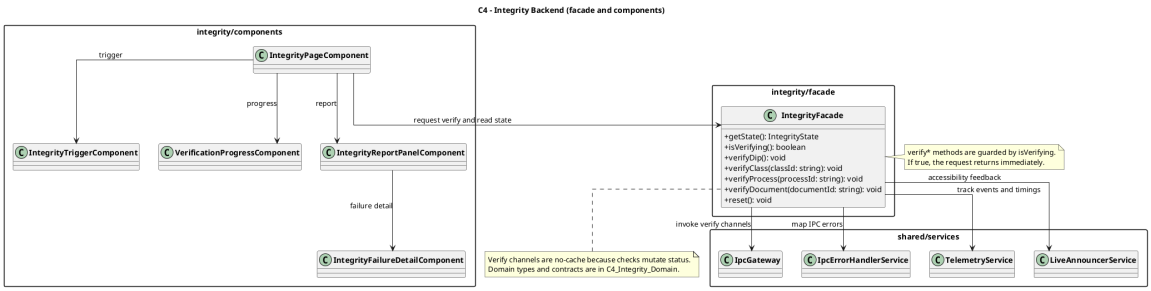

Figure 22: C4 del modulo Verifica Integrita - facade e componenti

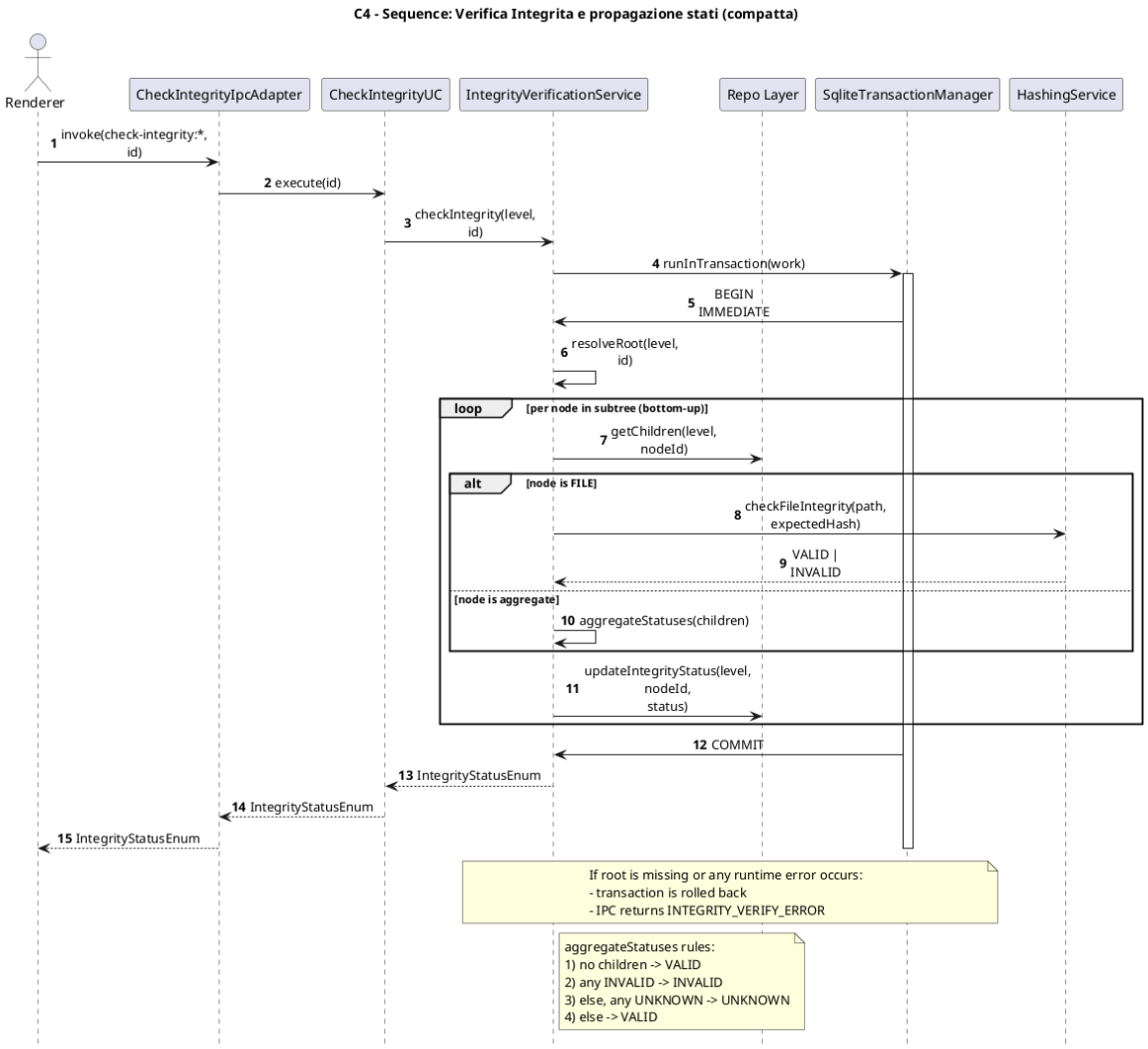

Figure 23: Sequenza di verifica e propagazione degli stati

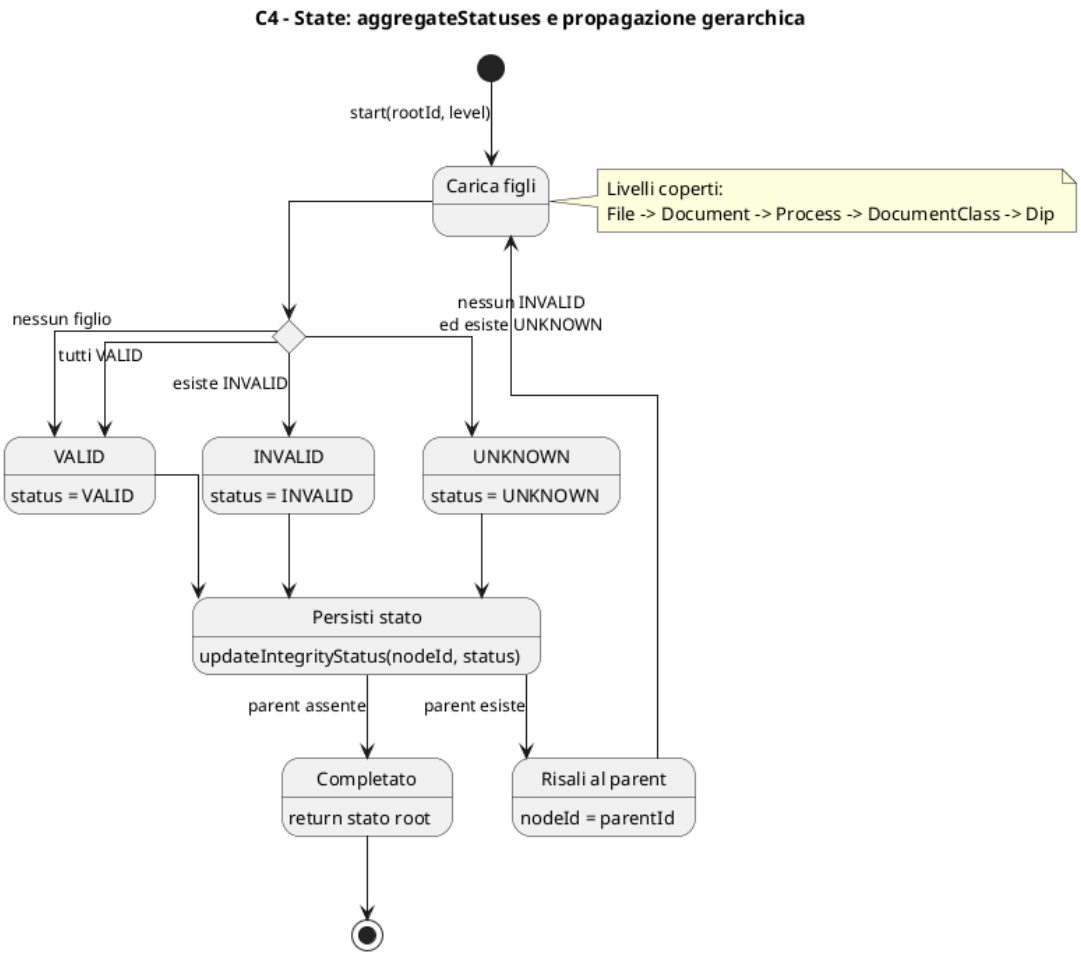

Figure 24: Macchina a stati per l'algoritmo di aggregazione

8.1.5 Use Case: firme check

Caratteristica della layer

I use case sono thin wrapper senza logica aggiuntiva; delegano direttamente a `IntegrityVerificationService`.

Use Case	Firma
ICheckFileIntegrityStatusUC	execute(fileId: number): Promise<IntegrityStatusEnum>
ICheckDocumentIntegrityStatusUC	execute(documentId: number): Promise<IntegrityStatusEnum>
ICheckProcessIntegrityStatusUC	execute(processId: number): Promise<IntegrityStatusEnum>
ICheckDocumentClassIntegrityStatusUC	execute(documentClassId: number): Promise<IntegrityStatusEnum>
ICheckDipIntegrityStatusUC	execute(dipId: number): Promise<IntegrityStatusEnum>

Table 27: Firme use case Verifica Integrita

8.1.6 Servizio centrale

File: `IntegrityVerificationService.ts`.

Dipendenze iniettate.

1. `IFileRepository`
2. `IDocumentRepository`
3. `IProcessRepository`
4. `IDocumentClassRepository`
5. `IDipRepository`
6. `IHashingService`
7. `ITransactionManager`
8. `DIP_PATH_TOKEN`

Regole operative.

1. Ogni API pubblica gira dentro `runInTransaction`
2. Se la root non esiste, viene lanciato errore
3. La verifica e ricorsiva nel sottoalbero
4. Ogni livello aggiorna `integrity_status` via repository

8.1.7 Regole di aggregazione

Algoritmo aggregateStatuses

Caso 1: nessun figlio → `VALID`
Caso 2: almeno un figlio `INVALID` → `INVALID`
Caso 3: nessun `INVALID` ma almeno un `UNKNOWN` → `UNKNOWN`
Caso 4: tutti `VALID` → `VALID`

Metodo	Aggiornamenti
`checkFileIntegrityStatus`	Verifica hash file; aggiorna solo file
`checkDocumentIntegrityStatus`	Verifica file del documento; aggiorna file + documento
`checkProcessIntegrityStatus`	Verifica documenti e file del processo; aggiorna file + documento + processo
`checkDocumentClassIntegrityStatus`	Verifica processi/documenti/file della classe; aggiorna livelli inferiori + classe
`checkDipIntegrityStatus`	Verifica intero sottoalbero; aggiorna file + documento + processo + classe + DIP

Table 29: Semantica operativa per livello

Semantica per livello.

8.1.8 Hashing e transazioni nel modulo

HashingService. `checkFileIntegrity(filePath, expectedHash)` legge bytes, calcola SHA-256, converte digest Base64 e confronta con l'hash atteso.

Path assoluto file. Il service risolve il percorso con `path.resolve(dipPath, file.getPath())` prima della verifica hash.

Transaction manager. L'esecuzione avviene in `runInTransaction(work)`; in caso di errore durante la cascata e garantito rollback completo.

8.1.9 Layer Repository e DAO

Componente	Metodi rilevanti per verifica
`IFileRepository / FileDAO`	`getById`, `getByDocumentId`, `updateIntegrityStatus`
`IDocumentRepository / DocumentDAO`	`getById`, `getByProcessId`, `updateIntegrityStatus`
`IProcessRepository / ProcessDAO`	`getById`, `getByDocumentClassId`, `updateIntegrityStatus`
`IDocumentClassRepository / DocumentClassDAO`	`getById`, `getByDipId`, `updateIntegrityStatus`
`IDipRepository / DipDAO`	`getById`, `updateIntegrityStatus`

Table 31: Repository e DAO coinvolti nella verifica integrita

8.1.10 Comportamenti runtime

Semantica runtime

Root non trovato: eccezione su id non presente
Persistenza: aggiornamento `integrity_status` su tutti i livelli coinvolti
Atomicita: rollback completo su errore interno
Return IPC: stato finale `IntegrityStatusEnum`